

海南天然橡胶产业集团股份有限公司投资价值分析报告

MF 班-王钧

学号：2025281050217

一、公司背景介绍

1.1 公司概况

海南天然橡胶产业集团股份有限公司（股票简称：海南橡胶，股票代码：601118.SH）是于 2005 年 3 月成立的国有控股上市公司，总部位于海南省海口市。公司于 2011 年 1 月在上海证券交易所主板正式挂牌上市，成为中国资本市场唯一的天然橡胶种植、加工、贸易全产业链上市企业。公司控股股东为海南省农垦投资控股集团有限公司，实际控制人为海南省国有资产监督管理委员会。

1.2 核心定位与战略地位

公司是国家天然橡胶资源安全供给的重要保障者。天然橡胶作为重要的工业原料和战略物资，广泛应用于轮胎、军工、医疗等领域。海南橡胶承载着保障国内天然橡胶供给安全、稳定橡胶市场价格、参与国际橡胶市场定价的核心使命。在中国天然橡胶产业中，公司处于绝对龙头地位，是国内最大的天然橡胶生产企业。

1.3 主营业务与产业布局

公司构建了从“种、管、养、割”到初加工、深加工、贸易、研发的完整产业链。

种植与原料端：公司拥有国内最大的天然橡胶种植资源。根据公司年报等公开信息，其在海南岛拥有约 341 万亩橡胶林（其中自营约 203 万亩，通过“公司+合作社+农户”等形式控制约 138 万亩），并在云南、上海等地设有分支机构。这部分资产是公司最核心的资源和价值基础。

加工与产品端：公司拥有多家橡胶初加工厂，生产全乳胶（SCR WF）等标准胶以及浓缩乳胶等初级产品。同时，公司积极向高附加值产品延伸，产品线涵盖橡胶木、环保轮胎专用胶、特种橡胶（如航空轮胎胶）等。

贸易与供应链端：公司利用自身资源优势和上市平台，在全球范围内开展天然橡胶贸易业务，整合国内国际资源，形成了强大的采购、销售和物流网络。旗下拥有上海龙橡、新加坡 R1 公司等贸易平台，显著提升了市场影响力和议价能力。

科技研发与创新：公司设立国家级技术中心，专注于橡胶树新品种培育、割胶技术革新、高性能橡胶材料研发以及智慧橡胶园建设，致力于提升产业效率与产品竞争力。

1.4 发展历程与政策背景

公司前身可追溯至新中国成立初期建立的海南农垦，具有深厚的产业历史积淀。成立股份公司并上市，是海南农垦市场化改革的关键一步。公司的发展深度契合国家以下战略方向：

国家战略物资保障：始终肩负保障国家天然橡胶供给安全的责任。

海南自贸港建设：作为海南省属重要国有企业，公司积极参与自贸港建设，利用政策优势拓展国际贸易、加工增值、跨境资金流动等业务。

乡村振兴与农业现代化：通过橡胶产业带动海南当地大量农户就业与增收，并推动传统农业向标准化、智能化转型。

海南天然橡胶产业集团股份有限公司是一家以保障国家战略物资供给为核心使命，拥有稀缺土地与橡胶林资源，具备完整产业链的国有控股上市公司。其价值根植于国内最大的天然橡胶种植基地，并在此基础上拓展贸易与加工业务。公司的未来发展，既受益于海南自贸港的政策红利和对橡胶资源战略价值的重估，也持续面临国际胶价波动、成本上升及气候变

化等严峻挑战。其投资价值分析需重点权衡其资源战略属性、全产业链协同效应与周期波动性风险之间的平衡。

二、公司市场表现分析

2.1 股票价格的获取

本报告采用 AKShare 库获取海南橡胶 2022 年 1 月 1 日至 2024 年 12 月 31 日的股票交易数据，包括每日的开盘价、收盘价、最高价、最低价、成交量和成交额。获取的股票数据覆盖了 2022-2024 年完整的市场周期，能够充分反映公司股票在不同经济环境下的表现。

2.2 代码

```
import pandas as pd
import numpy as np
import akshare as ak
import pdfplumber
import os
import re
from collections import Counter
import jieba
import matplotlib.pyplot as plt
from wordcloud import WordCloud
from collections import Counter

# 导入可视化库
import matplotlib.pyplot as plt
import seaborn as sns

# 设置中文字体
plt.rcParams['font.family'] = 'sans-serif' # 全局字体类别
plt.rcParams['font.sans-serif'] = ['SimHei'] # 具体字体：黑体
plt.rcParams['axes.unicode_minus'] = False # 解决负号无法显示的问题

# 清洗股票数据

# 1. 只保留日期 和 收盘价
price_clean = hist_df[["日期", "收盘"]].copy()

# 2. 日期转为 datetime
price_clean["日期"] = pd.to_datetime(price_clean["日期"])

# 3. 按日期排序
price_clean = price_clean.sort_values("日期")

# 4. 设置日期为索引
price_clean = price_clean.set_index("日期")

# 5. 重命名字段
price_clean = price_clean.rename(columns={"收盘": "close"})

# 6. 基础检查
print(price_clean.head())
print(price_clean.tail())
print(price_clean.index.is_monotonic_increasing)
print(f"价格序列长度: {len(price_clean)}")

Last executed at 2025-12-14 11:20:00 in 15ms
```

close	
日期	
2020-01-02	4.99
2020-01-03	5.11
2020-01-06	5.16
2020-01-07	5.15
2020-01-08	5.01
close	
日期	
2024-12-25	5.41
2024-12-26	5.41
2024-12-27	5.56
2024-12-30	5.50
2024-12-31	5.41
True	
价格序列长度: 1212	

2.2 股票收益和波动计算

```
import pandas as pd
import numpy as np

# 1. 首先读取数据并查看列名
price_clean = pd.read_csv('stock_data.csv')
print("数据列名:", price_clean.columns.tolist())
print("\n数据前5行:")
print(price_clean.head())

# 2. 找出收盘价列的实际名称
# 常见的收盘价列名可能有: 'Close', '收盘价', '收盘', '价格', 'cClose' 等
# 我们列出一些可能的名称来查找
possible_close_names = ['close', 'Close', '收盘价', '收盘', '收盘价格', 'price', 'Price', 'CLOSE']
close_column = None

for col in possible_close_names:
    if col in price_clean.columns:
        close_column = col
        print(f"\n找到收盘价列: '{close_column}'")
        break

if close_column is None:
    print("\n警告: 未找到标准的收盘价列名")
    print("请从以下列名中选择一个作为收盘价列:")
    print(price_clean.columns.tolist())
    # 你可以手动指定, 比如: close_column = '收盘'
    close_column = input("请输入收盘价列名: ")

# 3. 找出日期列的实际名称
# 常见的日期列名可能有: 'date', 'Date', 'datetime', '时间', '交易日期' 等
possible_date_names = ['date', 'Date', 'datetime', '时间', '交易日期', '日期', 'timestamp', '时间戳']
date_column = None
```

```
for col in possible_date_names:
    if col in price_clean.columns:
        date_column = col
        print(f"找到日期列: '{date_column}'")
        break

if date_column is None:
    print("\n警告: 未找到标准的日期列名")
    print("请从以下列名中选择一个作为日期列:")
    print(price_clean.columns.tolist())
    date_column = input("请输入日期列名: ")

# 4. 设置日期索引
if date_column in price_clean.columns:
    price_clean[date_column] = pd.to_datetime(price_clean[date_column])
    price_clean = price_clean.set_index(date_column).sort_index()
else:
    print("使用默认索引")
    price_clean = price_clean.sort_index()

# 5. 过滤日期范围为2022-01-01到2024-12-31
try:
    price_clean = price_clean.loc['2022-01-01':'2024-12-31']
    print(f"\n过滤后数据日期范围: {price_clean.index.min()} 到 {price_clean.index.max()}")
except:
    print("\n注意: 数据日期可能不在此范围内")

print(f"\n数据形状: {price_clean.shape}")
```

```
# 6. 现在计算对数收益率
final_df = price_clean.copy().sort_index()

# 使用找到的收盘价列名
final_df["log_ret"] = np.log(final_df[close_column]).diff()

# 7. 查看结果 (只显示相关列)
display_columns = [close_column, 'log_ret']
if 'open' in final_df.columns:
    display_columns.insert(0, 'open')
if 'high' in final_df.columns:
    display_columns.insert(1, 'high')
if 'low' in final_df.columns:
    display_columns.insert(2, 'low')

print("\n计算结果 (前10行):")
print(final_df[display_columns].head(10))

print("\n对数收益率统计:")
print(final_df["log_ret"].describe())
```

Last executed at 2025-12-14 15:28:11 in 49ms

数据列名: ['日期', '股票代码', '开盘', '收盘', '最高', '最低', '成交量', '成交额', '振幅', '涨跌幅', '涨跌额', '换手率']

数据前5行:

	日期	股票代码	开盘	收盘	最高	最低	成交量 \
0	2020-01-02	601118	4.960000	4.990000	5.020000	4.920000	291713
1	2020-01-03	601118	5.010000	5.110000	5.140000	4.990000	391291
2	2020-01-06	601118	5.100000	5.160000	5.200000	5.050000	396763
3	2020-01-07	601118	5.140000	5.150000	5.200000	5.110000	253160
4	2020-01-08	601118	5.150000	5.010000	5.150000	4.970000	349116

	成交额	振幅	涨跌幅	涨跌额	换手率
0	145038349.000000	2.030000	1.420000	0.070000	0.680000
1	198457858.000000	3.010000	2.400000	0.120000	0.910000
2	203428242.000000	2.940000	0.980000	0.050000	0.930000
3	130372855.000000	1.740000	-0.190000	-0.010000	0.590000
4	175936261.000000	3.500000	-2.720000	-0.140000	0.820000

找到收盘价列: '收盘'

找到日期列: '日期'

过滤后数据日期范围: 2022-01-04 00:00:00 到 2024-12-31 00:00:00

数据形状: (726, 11)

计算结果 (前10行):

	收盘	log_ret
日期		
2022-01-04	5.380000	NaN
2022-01-05	5.340000	-0.007463
2022-01-06	5.280000	-0.011300
2022-01-07	5.300000	0.003781
2022-01-10	5.350000	0.009390
2022-01-11	5.310000	-0.007505
2022-01-12	5.350000	0.007505
2022-01-13	5.350000	0.000000
2022-01-14	5.230000	-0.022685
2022-01-17	5.220000	-0.001914

对数收益率统计:

```
count    725.000000
mean      0.000008
std       0.017626
min      -0.069642
25%      -0.009132
50%       0.000000
75%       0.008791
max       0.096247
Name: log_ret, dtype: float64
```

```
# 计算波动率

# 2. 计算 20 日滚动年化波动率
window = 20 # 20 个交易日
trading_days = 252 # 一年交易日数

# 20日滚动年化波动率
window = 20

final_df["volatility"] = (
    final_df["log_ret"]
    .rolling(window)
    .std()
    * np.sqrt(trading_days)
)

# 看一眼结果
final_df.head(25)
```

Last executed at 2025-12-14 15:28:40 in 30ms

日期	股票代码	开盘	收盘	最高	最低	成交量	成交额	振幅	涨跌幅	涨跌额	换手率	log_ret	波动率
2022-01-04	601118	5.280000	5.380000	5.390000	5.260000	504167	269798880.000000	2.460000	1.890000	0.100000	1.180000	NaN	N
2022-01-05	601118	5.400000	5.340000	5.450000	5.330000	392050	210870111.000000	2.230000	-0.740000	-0.040000	0.920000	-0.007463	N
2022-01-06	601118	5.320000	5.280000	5.360000	5.280000	289453	153491457.000000	1.500000	-1.120000	-0.060000	0.680000	-0.011300	N
2022-01-07	601118	5.290000	5.300000	5.350000	5.280000	337198	179135206.000000	1.330000	0.380000	0.020000	0.790000	0.003781	N
2022-01-10	601118	5.290000	5.350000	5.370000	5.270000	294437	156926376.000000	1.890000	0.940000	0.050000	0.690000	0.009390	N
2022-01-11	601118	5.350000	5.310000	5.360000	5.290000	241358	128430786.000000	1.310000	-0.750000	-0.040000	0.560000	-0.007505	N
2022-01-12	601118	5.310000	5.350000	5.360000	5.290000	246077	130941912.000000	1.320000	0.750000	0.040000	0.580000	0.007505	N
2022-01-13	601118	5.360000	5.350000	5.440000	5.330000	347180	186457201.000000	2.060000	0.000000	0.000000	0.810000	0.000000	N
2022-01-14	601118	5.330000	5.230000	5.340000	5.210000	306236	161405181.000000	2.430000	-2.240000	-0.120000	0.720000	-0.022685	N
2022-01-17	601118	5.230000	5.220000	5.290000	5.190000	275455	144045006.000000	1.910000	-0.190000	-0.010000	0.640000	-0.001914	N
2022-01-18	601118	5.190000	5.180000	5.200000	5.120000	322091	165836309.000000	1.530000	-0.770000	-0.040000	0.750000	-0.007692	N
2022-01-19	601118	5.180000	5.150000	5.210000	5.100000	246911	127384812.000000	2.120000	-0.580000	-0.030000	0.580000	-0.005808	N
2022-01-20	601118	5.150000	5.100000	5.180000	5.090000	254979	130635101.000000	1.750000	-0.970000	-0.050000	0.600000	-0.009756	N
2022-01-21	601118	5.060000	5.020000	5.080000	4.990000	306460	153688671.000000	1.760000	-1.570000	-0.080000	0.720000	-0.015811	N
2022-01-24	601118	4.960000	4.970000	5.010000	4.950000	157826	78556227.000000	1.200000	-1.000000	-0.050000	0.370000	-0.010010	N
2022-01-25	601118	4.980000	4.780000	4.990000	4.770000	332585	161719006.000000	4.430000	-3.820000	-0.190000	0.780000	-0.038979	N
2022-01-26	601118	4.800000	4.830000	4.850000	4.780000	162990	78514123.000000	1.460000	1.050000	0.050000	0.380000	0.010406	N
2022-01-27	601118	4.820000	4.760000	4.850000	4.760000	198373	95184134.000000	1.860000	-1.450000	-0.070000	0.460000	-0.014599	N
2022-01-28	601118	4.750000	4.830000	4.870000	4.730000	177336	85226939.000000	2.940000	1.470000	0.070000	0.410000	0.014599	N
2022-02-07	601118	4.940000	4.900000	4.950000	4.890000	180888	88875111.000000	1.240000	1.450000	0.070000	0.420000	0.014389	N
2022-02-08	601118	4.910000	5.020000	5.030000	4.890000	232070	115681156.000000	2.860000	2.450000	0.120000	0.540000	0.024195	0.2320
2022-02-09	601118	5.010000	5.140000	5.190000	5.010000	410525	210240902.000000	3.590000	2.390000	0.120000	0.960000	0.023623	0.2500
2022-02-10	601118	5.180000	5.200000	5.230000	5.120000	309749	160436750.000000	2.140000	1.170000	0.060000	0.720000	0.011606	0.2520
2022-02-11	601118	5.200000	5.170000	5.250000	5.150000	285578	148216212.000000	1.920000	-0.580000	-0.030000	0.670000	-0.005786	0.2520
2022-02-14	601118	5.160000	5.180000	5.210000	5.130000	227931	117865187.000000	1.550000	0.190000	0.010000	0.530000	0.001932	0.2490

```
# 可视化，先找出高波动的阶段

# 高波动标记 (80%分位)
threshold = final_df["volatility"].quantile(0.8)
# 标记是否处于高波动阶段
final_df["high_vol"] = final_df["volatility"] > threshold

# 看一眼结果
final_df.head(30)

Last executed at 2025-12-14 15:29:04 in 22ms
```

日期	股票代码	开盘	收盘	最高	最低	成交量	成交额	振幅	涨跌幅	涨跌额	换手率	log_ret	波动率
2022-01-04	601118	5.280000	5.380000	5.390000	5.260000	504167	269798880.000000	2.460000	1.890000	0.100000	1.180000	NaN	N
2022-01-05	601118	5.400000	5.340000	5.450000	5.330000	392050	210870111.000000	2.230000	-0.740000	-0.040000	0.920000	-0.007463	N
2022-01-06	601118	5.320000	5.280000	5.360000	5.280000	289453	153491457.000000	1.500000	-1.120000	-0.060000	0.680000	-0.011300	N
2022-01-07	601118	5.290000	5.300000	5.350000	5.280000	337198	179135206.000000	1.330000	0.380000	0.020000	0.790000	0.003781	N
2022-01-10	601118	5.290000	5.350000	5.370000	5.270000	294437	156926376.000000	1.890000	0.940000	0.050000	0.690000	0.009390	N
2022-01-11	601118	5.350000	5.310000	5.360000	5.290000	241358	128430786.000000	1.310000	-0.750000	-0.040000	0.560000	-0.007505	N
2022-01-12	601118	5.310000	5.350000	5.360000	5.290000	246077	130941912.000000	1.320000	0.750000	0.040000	0.580000	0.007505	N
2022-01-13	601118	5.360000	5.350000	5.440000	5.330000	347180	186457201.000000	2.060000	0.000000	0.000000	0.810000	0.000000	N
2022-01-14	601118	5.330000	5.230000	5.340000	5.210000	306236	161405181.000000	2.430000	-2.240000	-0.120000	0.720000	-0.022685	N
2022-01-17	601118	5.230000	5.220000	5.290000	5.190000	275455	144045006.000000	1.910000	-0.190000	-0.010000	0.640000	-0.001914	N
2022-01-18	601118	5.190000	5.180000	5.200000	5.120000	322091	165836309.000000	1.530000	-0.770000	-0.040000	0.750000	-0.007692	N
2022-01-19	601118	5.180000	5.150000	5.210000	5.100000	246911	127384812.000000	2.120000	-0.580000	-0.030000	0.580000	-0.005808	N
2022-01-20	601118	5.150000	5.100000	5.180000	5.090000	254979	130635101.000000	1.750000	-0.970000	-0.050000	0.600000	-0.009756	N
2022-01-21	601118	5.060000	5.020000	5.080000	4.990000	306460	153688671.000000	1.760000	-1.570000	-0.080000	0.720000	-0.015811	N
2022-01-24	601118	4.960000	4.970000	5.010000	4.950000	157826	78556227.000000	1.200000	-1.000000	-0.050000	0.370000	-0.010010	N
2022-01-25	601118	4.980000	4.780000	4.990000	4.770000	332585	161719006.000000	4.430000	-3.820000	-0.190000	0.780000	-0.038979	N
2022-01-26	601118	4.800000	4.830000	4.850000	4.780000	162990	78514123.000000	1.460000	1.050000	0.050000	0.380000	0.010406	N
2022-01-27	601118	4.820000	4.760000	4.850000	4.760000	198373	95184134.000000	1.860000	-1.450000	-0.070000	0.460000	-0.014599	N
2022-01-28	601118	4.750000	4.830000	4.870000	4.730000	177336	85226939.000000	2.940000	1.470000	0.070000	0.410000	0.014599	N
2022-02-07	601118	4.940000	4.900000	4.950000	4.890000	180888	88875111.000000	1.240000	1.450000	0.070000	0.420000	0.014389	N
2022-02-08	601118	4.910000	5.020000	5.030000	4.890000	232070	115681156.000000	2.860000	2.450000	0.120000	0.540000	0.024195	0.2320
2022-02-09	601118	5.010000	5.140000	5.190000	5.010000	410525	210240902.000000	3.590000	2.390000	0.120000	0.960000	0.023623	0.2500
2022-02-10	601118	5.180000	5.200000	5.230000	5.120000	309749	160436750.000000	2.140000	1.170000	0.060000	0.720000	0.011606	0.2520
2022-02-11	601118	5.200000	5.170000	5.250000	5.150000	285578	148216212.000000	1.920000	-0.580000	-0.030000	0.670000	-0.005786	0.2520
2022-02-14	601118	5.160000	5.180000	5.210000	5.130000	227931	117865187.000000	1.550000	0.190000	0.010000	0.530000	0.001932	0.2490
2022-02-15	601118	5.160000	5.130000	5.180000	5.080000	240463	123152926.000000	1.930000	-0.970000	-0.050000	0.560000	-0.009699	0.2500
2022-02-16	601118	5.130000	5.150000	5.170000	5.110000	152990	78603176.000000	1.170000	0.390000	0.020000	0.360000	0.003891	0.2480
2022-02-17	601118	5.130000	5.100000	5.150000	5.090000	133211	68107922.000000	1.170000	-0.970000	-0.050000	0.310000	-0.009756	0.2500
2022-02-18	601118	5.080000	5.160000	5.160000	5.070000	136496	69924473.000000	1.760000	1.180000	0.060000	0.320000	0.011696	0.2420
2022-02-21	601118	5.150000	5.160000	5.170000	5.110000	143605	73867596.000000	1.160000	0.000000	0.000000	0.340000	0.000000	0.2420

```

import matplotlib.pyplot as plt
import pandas as pd
import numpy as np

# 确保final_df已经计算了log_ret
df = final_df.copy()

# 计算20日滚动年化波动率
rolling_window = 20
df['rolling_vol'] = df['log_ret'].rolling(window=rolling_window).std() * np.sqrt(252)

# 识别高波动阶段（例如波动率超过75%分位数）
vol_threshold = df['rolling_vol'].quantile(0.75) # 75%分位数作为阈值
high_vol_periods = df['rolling_vol'] > vol_threshold

# 创建图表
fig, ax1 = plt.subplots(figsize=(14, 7))

# 左轴：对数收益率（蓝色）
color1 = 'blue'
ax1.plot(df.index, df['log_ret'], color=color1, linewidth=1.5, label='对数收益率', alpha=0.8)
ax1.axhline(y=0, color='gray', linestyle='--', linewidth=0.8, alpha=0.5)
ax1.set_xlabel('日期', fontsize=12)
ax1.set_ylabel('对数收益率', color=color1, fontsize=12)
ax1.tick_params(axis='y', labelcolor=color1)
ax1.grid(True, alpha=0.3)
ax1.legend(loc='upper left')

# 右轴：20日滚动年化波动率（红色）
ax2 = ax1.twinx()
color2 = 'red'
ax2.plot(df.index, df['rolling_vol'], color=color2, linewidth=2, label='20日滚动年化波动率', alpha=0.8)
ax2.set_ylabel('20日滚动年化波动率', color=color2, fontsize=12)
ax2.tick_params(axis='y', labelcolor=color2)

# 高亮高波动阶段（在背景添加阴影）
if high_vol_periods.any():
    # 找到连续的高波动区间
    in_high_vol = False
    start_idx = None

    for i, (idx, is_high) in enumerate(zip(df.index, high_vol_periods)):
        if is_high and not in_high_vol:
            in_high_vol = True
            start_idx = idx
        elif not is_high and in_high_vol:
            in_high_vol = False
            ax1.axvspan(start_idx, df.index[i-1], alpha=0.2, color='orange', label='高波动阶段' if i==1 else "")

    # 处理最后一个区间
    if in_high_vol:
        ax1.axvspan(start_idx, df.index[-1], alpha=0.2, color='orange')

# 设置标题
plt.title('收益率与波动率（高波动阶段高亮）', fontsize=16, fontweight='bold', pad=20)

# 合并图例
lines1, labels1 = ax1.get_legend_handles_labels()
lines2, labels2 = ax2.get_legend_handles_labels()
ax1.legend(lines1 + lines2, labels1 + labels2, loc='upper center', bbox_to_anchor=(0.5, -0.12), ncol=3)

# 调整布局
plt.tight_layout()
plt.show()

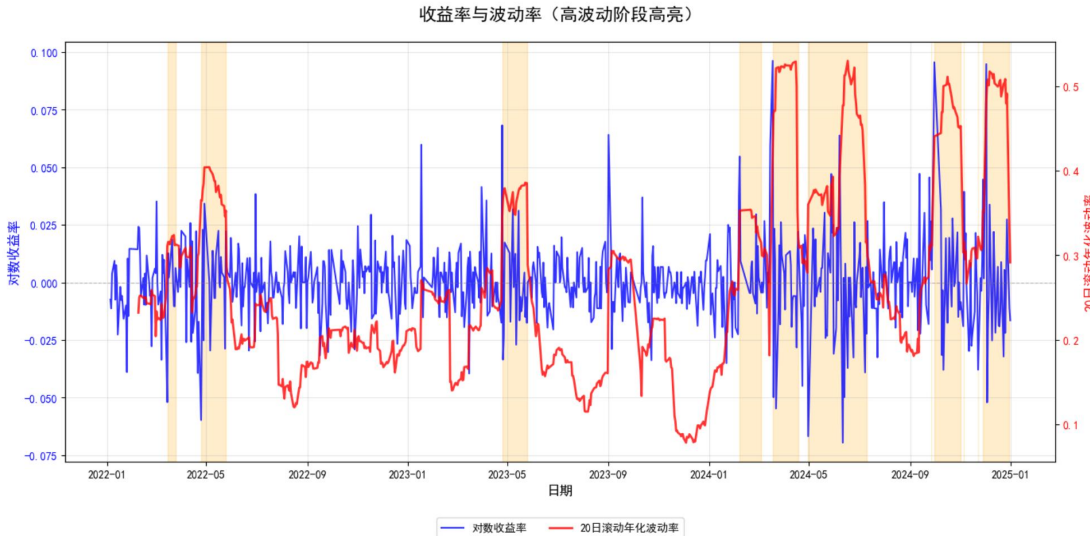
```

```

# 打印统计信息
print("统计信息：")
print("=" * 50)
print(f"数据期间：{df.index.min().date()} 至 {df.index.max().date()}")
print(f"收益率均值：{df['log_ret'].mean():.6f}")
print(f"收益率标准差：{df['log_ret'].std():.6f}")
print(f"20日滚动波动率均值：{df['rolling_vol'].mean():.4f}")
print(f"高波动阈值（75%分位数）：{vol_threshold:.4f}")
print(f"高波动天数：{high_vol_periods.sum()} / {len(df)} ({high_vol_periods.sum()/len(df)*100:.1f}%")

最后执行时间为2025-12-14, 时间15: 33: 59, 耗时307毫秒

```



统计信息:

数据期间: 2022-01-04 至 2024-12-31
收益率均值: 0.000008
收益率标准差: 0.017626
20日滚动波动率均值: 0.2613
高波动阈值 (75%分位数): 0.3118
高波动天数: 177 / 726 (24.4%)

2.3 分析

根据收益率与波动率图表，对该公司的经营和市场表现分析如下：

2.3.1 波动特征分析

三个阶段识别：

（1）剧烈波动期（2022 年初-2023 年初）

波动率持续高于 0.3（年化 30%以上），收益率在 $\pm 7.5\%$ 间大幅震荡。

经营推断：面临重大不确定性（疫情后复苏、成本冲击或战略调整）

（2）平稳过渡期（2023 年中-2024 年中）

波动率下降至 0.2 以下，收益率振幅收窄至 $\pm 2.5\%$ 。

经营推断：业务进入稳定期，市场预期趋于一致。

（3）再度波动期（2024 年中-2025 年初）

波动率回升至 0.3-0.4 区间，出现多次 $\pm 5\%$ 以上的单日波动。

经营推断：面临新一轮挑战或转型（行业周期、竞争加剧或新业务投入）。

2.3.2 经营质量关键信号

积极信号：

波动率有回落能力：显示公司具备经营韧性。

无持续性单边下跌：收益率分布相对均衡。

风险信号：

波动率基线抬升：2024 年后波动中枢高于 2023 年。

收益率尖峰频繁：短期大幅波动反映信息敏感度高。

2.3.3 经营情况推断

（1）周期性敏感：波动与宏观经济/行业周期高度相关。

（2）转型期特征：2024 年波动回升可能对应业务结构调整。

（3）盈利波动性：收益率大幅摆动暗示盈利预期频繁调整。

2.3.4 结论

该公司经营具有显著的周期性与转型期特征。2023 年的平稳期显示公司能适应环境变化，但 2024 年波动率再度上升需重点关注——是短期扰动还是结构性变化的开始。建议结合该时期的财务报告与行业动态，判断波动驱动因素是外部环境还是内部经营调整。

三、公司财务绩效分析

3.1 三张表的获取代码

```
# 财务数据的报告期有3年
report_dates = [ "20221231", "20231231", "20241231" ]

Last executed at 2025-12-14 11:34:15 in 7ms

import akshare as ak
import pandas as pd

# 定义要获取的财务报告日期
# 通常A股年报的发布日期是每年4月底前，我们取年报截止日期
report_dates = [ "20221231", "20231231", "20241231" ] # 2024年报要到2025年4月底才发布

balance_list = []

for date in report_dates:
    print(f"正在下载 {date} 的资产负债表数据...")

    try:
        # 取"全市场"资产负债表
        df_all = ak.stock_zcfz_em(date=date)

        # 筛选海南橡胶（股票代码：601118）
        # 需要确认akshare返回的数据中股票代码的列名
        # 可能是'股票代码'、'代码'、'symbol'等

        # 尝试几种可能的列名
        possible_code_columns = [ '股票代码', '代码', 'symbol', '证券代码' ]
        code_column = None

        for col in possible_code_columns:
            if col in df_all.columns:
                code_column = col
                break

    if code_column:
        # 筛选海南橡胶
        df_601118 = df_all[df_all[code_column] == '601118']

        if not df_601118.empty:
            # 添加报告日期列
            df_601118['报告日期'] = date
            balance_list.append(df_601118)
            print(f" 成功获取 {date} 的资产负债表数据")
        else:
            print(f" 未找到 {date} 海南橡胶(601118)的数据")
        else:
            print(f" 无法识别 {date} 数据中的股票代码列，可用列名: {df_all.columns.tolist()}")

    except Exception as e:
        print(f" 下载 {date} 数据时出错: {e}")

# 合并所有数据
if balance_list:
    balance_df = pd.concat(balance_list, ignore_index=True)

    # 保存到文件
    filename = "hainan_rubber_balance_sheet_2022_2023.csv"
    balance_df.to_csv(filename, index=False, encoding='utf-8-sig')
    print(f"\n资产负债表数据已保存到: {filename}")
    print(f"\n数据预览:")
    print(balance_df.head())

    # 显示关键财务指标
    print("\n关键财务指标:")
    key_columns = [ '报告日期', '股票代码', '资产总计', '负债合计', '所有者权益合计' ]
```



```
# 显示关键财务指标
print("\n关键财务指标:")
key_columns = ['报告日期', '股票代码', '资产总计', '负债合计', '所有者权益合计']

# 根据实际列名调整
available_columns = [col for col in key_columns if col in balance_df.columns]
if available_columns:
    print(balance_df[available_columns].to_string(index=False))
else:
    print("未找到标准列名, 所有列如下:")
    print(balance_df.columns.tolist())
else:
    print("没有获取到任何资产负债表数据")

# 如果需要2024年的数据, 可以尝试其他方法
print("\n" + "="*50)
print("如果需要2024年数据, 请尝试以下方法:")

# 方法2: 使用单个股票接口
print("\n方法2: 直接获取单个股票财务数据")
try:
    # 获取海南橡胶的资产负债表
    stock_code = "601118"
    balance_single = ak.stock_balance_sheet_by_report_em(symbol=stock_code)

    if not balance_single.empty:
        # 筛选2022-2024年的年度报告
        # 先查看数据的结构和列名
        print(f"\n获取到 {stock_code} 的资产负债表数据, 列名:")
        print(balance_single.columns.tolist())

        # 根据实际数据结构筛选年度数据
        # 通常会有'REPORT_DATE'或类似列
        # 这里需要根据实际数据结构调整

        filename_single = f"{stock_code}_balance_sheet_all.csv"
        balance_single.to_csv(filename_single, index=False, encoding='utf-8-sig')
        print(f"\n单个股票数据已保存到: {filename_single}")

except Exception as e:
    print(f"获取单个股票数据时出错: {e}")
```

Last executed at 2025-12-14 13:52:59 in 50.39s

成功获取 20241231 的资产负债表数据

资产负债表数据已保存到: hainan_rubber_balance_sheet_2022_2023.csv

数据预览:

	序号	股票代码	股票简称	资产-货币资金	资产-应收账款	资产-存货	资产-总资产 \	
0	566	601118	海南橡胶	5.680445e+09	5.810297e+08	1.541119e+09	2.232898e+10	
1	1480	601118	海南橡胶	3.505677e+09	1.669754e+09	4.525802e+09	3.395033e+10	
2	512	601118	海南橡胶	5.687636e+09	2.259762e+09	5.224534e+09	3.606977e+10	
资产-总资产同比				负债-应付账款	负债-预收账款	负债-总负债	负债-总负债同比	资产负债率 \
0	15.147047	6.932899e+08	9.479763e+07	1.263685e+10	30.441551	56.593958		
1	52.046056	1.120149e+09	7.811795e+07	2.231586e+10	76.593512	65.730911		
2	6.242763	1.436412e+09	1.007029e+08	2.477138e+10	11.003492	68.676308		
股东权益合计			公告日期	报告日期				
0	9.692125e+09	2024-04-30	20221231					
1	1.163447e+10	2025-04-26	20231231					
2	1.129838e+10	2025-10-31	20241231					

关键财务指标:

报告日期	股票代码
20221231	601118
20231231	601118
20241231	601118

```
# 保存资产负债表 (5年)
balance_df.to_csv("balance_5y.csv",index=False,encoding="utf-8-sig")
print("资产负债表已保存: balance_5y.csv")
```

Last executed at 2025-12-14 13:54:48 in 12ms

资产负债表已保存: balance_5y.csv

```
import akshare as ak
import pandas as pd

# 定义股票代码和报告日期
stock_code = "601118"
report_dates = ["20221231", "20231231", "20241231"]

profit_list = []

for date in report_dates:
    print(f"正在下载 {date} 的利润表数据...")

    try:
        # 获取全市场利润表
        df_all = ak.stock_lrb_em(date=date)

        print(f" 数据形状: {df_all.shape}")
        print(f" 列名: {df_all.columns.tolist()}")

        # 先查看数据前几行, 了解数据结构
        print(f"\n 前3行数据:")
        print(df_all.head(3).to_string())
```

```
# 找到股票代码的列名
code_column = None
for col in df_all.columns:
    # 检查这一列是否包含股票代码
    if any(keyword in col for keyword in ['代码', '股票', 'symbol', '证券']):
        code_column = col
        print(f" 找到股票代码列: {code_column}")
        break

if code_column:
    # 将股票代码列转为字符串
    df_all[code_column] = df_all[code_column].astype(str)

    # 查看这一列的一些示例值
    print(f" 股票代码列前5个值: {df_all[code_column].head().tolist()}")

    # 筛选目标股票
    # 注意: akshare 可能返回带有市场前缀的代码, 如'601118.SH'或'SH601118'
    mask = (
        (df_all[code_column] == stock_code) |
        (df_all[code_column] == f"{stock_code}.SH") |
        (df_all[code_column] == f"SH{stock_code}") |
        (df_all[code_column] == f"{stock_code}.sh")
    )

    df_one = df_all[mask].copy()
```

```
if not df_one.empty:
    df_one["报告期"] = date
    profit_list.append(df_one)
    print(f" 成功获取 {stock_code} {date} 的利润表数据")
    print(f" 找到 {len(df_one)} 条记录")
else:
    print(f" 未找到 {stock_code} 的数据, 尝试其他匹配方式...")

    # 尝试部分匹配
    mask_partial = df_all[code_column].str.contains(stock_code)
    df_partial = df_all[mask_partial].copy()

    if not df_partial.empty:
        print(f" 通过部分匹配找到 {len(df_partial)} 条记录:")
        print(df_partial[code_column].unique())

        # 使用第一个匹配
        df_partial["报告期"] = date
        profit_list.append(df_partial.iloc[[0]])
        print(f" 使用第一条匹配记录")
    else:
        print(f" 完全未找到包含 {stock_code} 的记录")

else:
    print(f" 未找到明显的股票代码列")
    print(f" 所有列名: {df_all.columns.tolist()}")
```

```

# 尝试查找包含数字代码的列
for col in df_all.columns:
    sample_value = str(df_all[col].iloc[0]) if len(df_all) > 0 else ""
    if stock_code in sample_value:
        print(f" 在第 {col} 列找到包含 {stock_code} 的值: {sample_value}")
        code_column = col
        break

if code_column:
    df_all[code_column] = df_all[code_column].astype(str)
    mask = df_all[code_column].str.contains(stock_code)
    df_one = df_all[mask].copy()

    if not df_one.empty:
        df_one["报告期"] = date
        profit_list.append(df_one)
        print(f" 通过包含匹配找到数据")

except Exception as e:
    print(f" 处理 {date} 数据时出错: {e}")

# 合并数据
if profit_list:
    profit_df = pd.concat(profit_list, ignore_index=True)

# 保存到文件
filename = f"{stock_code}_profit_statement.csv"
profit_df.to_csv(filename, index=False, encoding='utf-8-sig')
print(f"\n利润表数据已保存到: {filename}")

```

```

# 显示数据
print(f"\n获取到的数据 ({len(profit_df)} 条记录:)")
print(profit_df.to_string(index=False))

# 提取关键字段
print(f"\n所有列名:")
for col in profit_df.columns:
    print(f" {col}")

else:
    print("没有获取到任何利润表数据")

# 方法2: 使用更简单的方法直接获取单个股票数据
print("\n" + "="*50)
print("方法2: 直接获取单个股票财务数据")

try:
    # 使用不同的接口
    print("\n尝试方法2.1: 使用stock_financial_report_sina")
    from datetime import datetime

    # 获取年度数据
    years = [2022, 2023]

    for year in years:
        print(f"\n获取 {year} 年数据...")
        try:
            # 新浪财经接口
            df_sina = ak.stock_financial_report_sina(stock=stock_code, symbol="利润表")

            if not df_sina.empty:
                print(f" 获取到数据, 形状: {df_sina.shape}")
                print(f" 列名: {df_sina.columns.tolist()}")

```



```
# 筛选年度数据
if '报表日期' in df_sina.columns:
    df_sina['报表日期'] = pd.to_datetime(df_sina['报表日期'])
    df_year = df_sina[df_sina['报表日期'].dt.year == year]

    if not df_year.empty:
        print(f"  找到 {year} 年数据:")
        print(df_year.to_string(index=False))

    # 保存数据
    filename = f"{stock_code}_profit_{year}.csv"
    df_year.to_csv(filename, index=False, encoding='utf-8-sig')
    print(f"  数据已保存到: {filename}")

except Exception as e:
    print(f"  获取 {year} 年数据时出错: {e}")

except Exception as e:
    print(f"方法2出错: {e}")

# 方法3: 使用东方财富的单个股票接口
print("\n" + "="*50)
print("方法3: 使用东方财富单个股票财务数据")

try:
    print(f"\n获取 {stock_code} 的利润表...")
    df_em = ak.stock_profit_sheet_by_yearly_em(symbol=stock_code)

    if not df_em.empty:
        print(f"获取成功, 数据形状: {df_em.shape}")
        print(f"\n列名:")
        for col in df_em.columns:
            print(f"  {col}")

    print(f"\n数据预览:")
    print(df_em.head().to_string())

    # 保存数据
    filename = f"{stock_code}_profit_yearly_em.csv"
    df_em.to_csv(filename, index=False, encoding='utf-8-sig')
    print(f"\n数据已保存到: {filename}")

    # 筛选2022-2023年数据
    if 'REPORT_DATE' in df_em.columns:
        df_em['年份'] = pd.to_datetime(df_em['REPORT_DATE']).dt.year
        df_recent = df_em[df_em['年份'].isin([2022, 2023])]

        if not df_recent.empty:
            print(f"\n2022-2023年数据:")
            print(df_recent.to_string(index=False))

except Exception as e:
    print(f"方法3出错: {e}")
    import traceback
    traceback.print_exc()

Last executed at 2025-12-14 14:01:22 in 56.23s
```

正在下载 20221231 的利润表数据...

显示小部件错误: 找不到模型

数据形状: (5263, 15)

列名: ['序号', '股票代码', '股票简称', '净利润', '净利润同比', '营业总收入', '营业总收入同比', '营业总支出-营业支出', '营业总支出-销售费用', '营业总支出-管理费用', '营业总支出-财务费用', '营业总支出-营业总支出', '营业利润', '利润总额', '公告日期']

前3行数据:

	序号	股票代码	股票简称	净利润	净利润同比	营业总收入	营业总收入同比	营业总支出-营业支出	营业总支出-销售费用
0	1	603322	超讯通信	1.518963e+07	106.580672	2.025843e+09	36.545295	1.857949e+09	34552197.20
e+09	-5.934288e+07	-4.422050e+07	2025-12-13						
1	2	301687	新广益	8.151343e+07	-2.379592	4.552627e+08	-8.232190	3.106639e+08	11794105.28
+08	9.111887e+07	9.094785e+07	2025-12-12						
2	3	301667	纳百川	1.133156e+08	134.646910	1.030915e+09	98.369944	8.062717e+08	12772325.50
+08	1.253271e+08	1.250819e+08	2025-12-12						

找到股票代码列: 股票代码

股票代码列前5个值: ['603322', '301687', '301667', '688809', '688802']

成功获取 601118 20221231 的利润表数据

找到 1 条记录

正在下载 20231231 的利润表数据...

显示小部件错误: 找不到模型

数据形状: (5220, 15)

列名: ['序号', '股票代码', '股票简称', '净利润', '净利润同比', '营业总收入', '营业总收入同比', '营业总支出-营业支出', '营业总支出-销售费用', '营业总支出-管理费用', '营业总支出-财务费用', '营业总支出-营业总支出', '营业利润', '利润总额', '公告日期']

前3行数据:

序号	股票代码	股票简称	净利润	净利润同比	营业总收入	营业总收入同比	营业总支出-营业支出	营业总支出-销售费用				
营业总支出-管理费用			营业总支出-财务费用	营业总支出-营业总支出	营业利润	利润总额	公告日期					
0	1	301687	新广益	83282545.63	2.170335	5.161417e+08	13.372283	3.508513e+08	11827761.02	24259897.80	-1338524.38	4.164846e+08
9.407036e+07			9.384962e+07	2025-12-12								
1	2	301667	纳百川	98254894.40	-13.290955	1.136217e+09	10.214387	9.185923e+08	14604849.43	46353506.22	-382467.83	1.029092e+09
1.083463e+08			1.064913e+08	2025-12-12								
2	3	688809	强一股份	18657720.58	19.429648	3.544391e+08	39.456691	1.899984e+08	13215240.36	30434070.66	-3651607.36	3.248633e+08
8			1.824317e+07	1.691008e+07	2025-12-11							

找到股票代码列: 股票代码

股票代码列前5个值: ['301687', '301667', '688809', '688802', '688790']

成功获取 601118 20231231 的利润表数据

找到 1 条记录

正在下载 20241231 的利润表数据...

显示小部件错误: 找不到模型

数据形状: (5199, 15)

列名: ['序号', '股票代码', '股票简称', '净利润', '净利润同比', '营业总收入', '营业总收入同比', '营业总支出-营业支出', '营业总支出-销售费用', '营业总支出-管理费用', '营业总支出-财务费用', '营业总支出-营业总支出', '营业利润', '利润总额', '公告日期']

前3行数据:

序号	股票代码	股票简称	净利润	净利润同比	营业收入	营业收入同比	营业总收入	营业总收入同比	营业总支出-营业支出	营业总支出-销售费	
用 营业总支出-管理费用			营业总支出-财务费用	营业总支出-营业总支出	营业收入	利润总额	公告日期				
0	1	301687 新广益	1.157019e+08	38.926988	6.571719e+08	27.323925	4.450090e+08	13427756.48	28405632.44	-1518039.62	5.202273
e+08			1.314234e+08	1.313780e+08	2025-12-12						
1	2	301667 纳百川	9.542884e+07	-2.876250	1.437053e+09	26.476980	1.187638e+09	17115074.97	45330720.64	10392611.96	1.323240
e+09			1.085188e+08	1.075983e+08	2025-12-12						
2	3	688809 强一股份	2.330970e+08	1149.332749	6.413604e+08	80.950808	2.458966e+08	13091937.81	34762622.06	1049997.59	3.81024
7e+08			2.586055e+08	2.575639e+08	2025-12-11						

找到股票代码列: 股票代码

股票代码列前5个值: ['301687', '301667', '688809', '688802', '688790']

成功获取 601118 20241231 的利润表数据

找到 1 条记录

利润表数据已保存到: 601118_profit_statement.csv

获取到的数据 (3 条记录):

序号	股票代码	股票简称	净利润	净利润同比	营业总收入	营业总收入同比	营业总支出-营业支出	营业总支出-销售费用	营业总支出-管理费用	营业总支出-财务费用	营业总支出-营业总支出	营业利润	利润总额	公告日期	报告期	
557	601118	海南橡胶	76399653.34	-49.325	1.537127e+10	1.077424	1.503802e+10	191363313.37	7.277051e+08	281060985.66	1.628981e+10	-2.555	146e+08	81795914.94	2024-04-30	20221231
1478	601118	海南橡胶	297124723.08	288.910	3.768725e+10	145.179792	3.688708e+10	232013552.50	1.549310e+09	673919517.49	3.948387e+10	-1.060	447e+09	320617342.50	2025-04-26	20231231
1478	601118	海南橡胶	103389020.00	-65.200	4.967261e+10	31.802151	4.802422e+10	314584246.34	1.568532e+09	691348334.86	5.076446e+10	-5.591	823e+08	135082036.04	2025-04-26	20241231

所有列名:

序号
股票代码
股票简称
净利润
净利润同比
营业总收入
营业总收入同比
营业总支出-营业支出
营业总支出-销售费用
营业总支出-管理费用
营业总支出-财务费用
营业总支出-营业总支出
营业利润
利润总额
公告日期
报告期

方法2: 直接获取单个股票财务数据

尝试方法2.1: 使用stock_financial_report_sina

获取 2022 年数据...

获取到数据, 形状: (76, 83)

列名: ['报告日', '营业总收入', '营业收入', '利息收入', '已赚保费', '手续费及佣金收入', '房地产销售收入', '其他业务收入', '营业总成本', '营业成本', '手续费及佣金支出', '房地产销售成本', '退保金', '赔付支出净额', '提取保险合同准备金净额', '保单红利支出', '分保费用', '其他业务成本', '营业税金及附加', '研发费用', '销售费用', '管理费用', '财务费用', '利息费用', '利息支出', '投资收益', '对联营企业和合营企业的投资收益', '以摊余成本计量的金融资产终止确认产生的收益', '汇兑收益', '净敞口套期收益', '公允价值变动收益', '期货损益', '托管收益', '补贴收入', '其他收益', '资产减值损失', '信用减值损失', '其他业务利润', '资产处置收益', '营业利润', '营业外收入', '非流动资产处置利得', '营业外支出', '非流动资产处置损失', '利润总额', '所得税费用', '未确认投资损失', '净利润', '持续经营净利润', '终止经营净利润', '归属于母公司所有者的净利润', '被合并方在合并前实现净利润', '少数股东损益', '其他综合收益', '归属于母公司所有者的其他综合收益', '(一)以后不能重分类进损益的其他综合收益', '重新计量设定受益计划变动额', '权益法下不能转损益的其他综合收益', '其他权益工具投资公允价值变动', '企业自身信用风险公允价值变动', '(二)以后将重分类进损益的其他综合收益', '权益法下可转损益的其他综合收益', '可供出售金融资产公允价值变动损益', '其他债权投资公允价值变动', '金融资产重分类计入其他综合收益的金额', '其他债权投资信用减值准备', '持有至到期投资重分类为可供出售金融资产损益', '现金流量套期储备', '现金流量套期损益的有效部分', '外币财务报表折算差额', '其他', '归属于少数股东的其他综合收益', '综合收益总额', '归属于母公司所有者的综合收益总额', '归属于少数股东的综合收益总额', '基本每股收益', '稀释每股收益', '数据源', '是否审计', '公告日期', '币种', '类型', '更新日期']

获取 2023 年数据...

获取到数据, 形状: (76, 83)

列名: ['报告日', '营业总收入', '营业收入', '利息收入', '已赚保费', '手续费及佣金收入', '房地产销售收入', '其他业务收入', '营业总成本', '营业成本', '手续费及佣金支出', '房地产销售成本', '退保金', '赔付支出净额', '提取保险合同准备金净额', '保单红利支出', '分保费用', '其他业务成本', '营业税金及附加', '研发费用', '销售费用', '管理费用', '财务费用', '利息费用', '利息支出', '投资收益', '对联营企业和合营企业的投资收益', '以摊余成本计量的金融资产终止确认产生的收益', '汇兑收益', '净敞口套期收益', '公允价值变动收益', '期货损益', '托管收益', '补贴收入', '其他收益', '资产减值损失', '信用减值损失', '其他业务利润', '资产处置收益', '营业利润', '营业外收入', '非流动资产处置利得', '营业外支出', '非流动资产处置损失', '利润总额', '所得税费用', '未确认投资损失', '净利润', '持续经营净利润', '终止经营净利润', '归属于母公司所有者的净利润', '被合并方在合并前实现净利润', '少数股东损益', '其他综合收益', '归属于母公司所有者的其他综合收益', '(一)以后不能重分类进损益的其他综合收益', '重新计量设定受益计划变动额', '权益法下不能转损益的其他综合收益', '其他权益工具投资公允价值变动', '企业自身信用风险公允价值变动', '(二)以后将重分类进损益的其他综合收益', '权益法下可转损益的其他综合收益', '可供出售金融资产公允价值变动损益', '其他债权投资公允价值变动', '金融资产重分类计入其他综合收益的金额', '其他债权投资信用减值准备', '持有至到期投资重分类为可供出售金融资产损益', '现金流量套期储备', '现金流量套期损益的有效部分', '外币财务报表折算差额', '其他', '归属于少数股东的其他综合收益', '综合收益总额', '归属于母公司所有者的综合收益总额', '归属于少数股东的综合收益总额', '基本每股收益', '稀释每股收益', '数据源', '是否审计', '公告日期', '币种', '类型', '更新日期']

```
import akshare as ak
import pandas as pd

# 定义股票代码和报告日期
stock_code = "601118"
report_dates = ["20221231", "20231231", "20241231"]

cashflow_list = []

for date in report_dates:
    print(f'正在下载 {date} 的现金流量表数据...')

    try:
        # 获取全市场现金流量表
        df_all = ak.stock_xjll_em(date=date)

        print(f" 数据形状: {df_all.shape}")
        print(f" 列名: {df_all.columns.tolist()}")

        # 找到股票代码的列名
        code_column = None
        possible_code_cols = ['股票代码', '代码', '证券代码', 'symbol', 'SECUCODE', 'S_INFO_WINDCODE']

        for col in possible_code_cols:
            if col in df_all.columns:
                code_column = col
                print(f" 找到股票代码列: {code_column}")
                break

        if code_column:
            # 将股票代码列转为字符串
            df_all[code_column] = df_all[code_column].astype(str)
```

```

# 查看股票代码列的示例值
print(f" 股票代码列前5个值: {df_all[code_column].head().tolist()}")

# 创建筛选条件
conditions = [
    df_all[code_column] == stock_code,
    df_all[code_column] == f"{stock_code}.SH",
    df_all[code_column] == f"SH{stock_code}",
    df_all[code_column] == f"{stock_code}.sh",
    df_all[code_column] == f"{stock_code}SH",
    df_all[code_column].str.contains(stock_code)
]

# 合并条件
mask = conditions[0]
for cond in conditions[1:]:
    mask = mask | cond

# 筛选数据
df_one = df_all[mask].copy()

if not df_one.empty:
    df_one["报告期"] = date
    cashflow_list.append(df_one)
    print(f" 成功获取 {stock_code} {date} 的现金流量表数据")
    print(f" 找到 {len(df_one)} 条记录")
else:
    print(f" 未找到 {stock_code} 的数据")

```

```

else:
    print(f" 未找到股票代码列")
    # 显示所有列名
    for col in df_all.columns:
        print(f" {col}")

except Exception as e:
    print(f" 处理 {date} 数据时出错: {e}")
    import traceback
    traceback.print_exc()

# 合并数据
if cashflow_list:
    cashflow_df = pd.concat(cashflow_list, ignore_index=True)

    # 保存到文件
    filename = f"{stock_code}_cashflow_statement.csv"
    cashflow_df.to_csv(filename, index=False, encoding='utf-8-sig')
    print(f"\n现金流量表数据已保存到: {filename}")

    # 显示数据
    print(f"\n获取到的数据:")
    print(cashflow_df.to_string(index=False))

else:
    print("没有获取到任何现金流量表数据")

# 方法2: 直接获取单个股票的现金流量表
print("\n" + "="*50)
print("方法2: 直接获取单个股票现金流量表数据")

```

```

try:
    print(f"\n获取 {stock_code} 的现金流量表...")

    # 使用东方财富接口
    df_cashflow = ak.stock_cash_flow_sheet_by_yearly_em(symbol=stock_code)

    if not df_cashflow.empty:
        print(f"获取成功, 数据形状: {df_cashflow.shape}")
        print(f"\n列名:")
        for col in df_cashflow.columns:
            print(f"    {col}")

        print(f"\n数据预览:")
        print(df_cashflow.head().to_string())

        # 保存所有数据
        filename_all = f"{stock_code}_cashflow_all_years.csv"
        df_cashflow.to_csv(filename_all, index=False, encoding='utf-8-sig')
        print(f"\n所有年度数据已保存到: {filename_all}")

        # 筛选2022-2023年数据
        if 'REPORT_DATE' in df_cashflow.columns:
            df_cashflow['REPORT_DATE'] = pd.to_datetime(df_cashflow['REPORT_DATE'])
            df_cashflow['年份'] = df_cashflow['REPORT_DATE'].dt.year

            df_recent = df_cashflow[df_cashflow['年份'].isin([2022, 2023])]

            if not df_recent.empty:
                print(f"\n2022-2023年现金流量表数据:")
                print(df_recent.to_string(index=False))

        # 保存最近三年数据
        filename_recent = f"{stock_code}_cashflow_2022_2024.csv"
        df_recent.to_csv(filename_recent, index=False, encoding='utf-8-sig')
        print(f"\n2022-2024年数据已保存到: {filename_recent}")

        # 显示关键指标
        print(f"\n关键现金流量指标:")
        key_columns = ['经营活动产生的现金流量净额', '投资活动产生的现金流量净额',
                       '筹资活动产生的现金流量净额', '现金及现金等价物净增加额']

        available_cols = [col for col in key_columns if col in df_cashflow.columns]
        if available_cols:
            print(df_cashflow[['年份'] + available_cols].to_string(index=False))

except Exception as e:
    print(f"方法2出错: {e}")
    import traceback
    traceback.print_exc()

# 方法3: 获取完整的财务报表
print("\n" + "="*50)
print("方法3: 获取完整财务报表")

try:
    # 获取财务指标摘要
    print(f"\n获取 {stock_code} 财务指标摘要...")
    df_financial = ak.stock_financial_abstract(symbol=stock_code)

    if not df_financial.empty:
        print(f"获取成功, 数据形状: {df_financial.shape}")

```

```
# 筛选年度数据
if 'REPORT_DATE' in df_financial.columns:
    df_financial['REPORT_DATE'] = pd.to_datetime(df_financial['REPORT_DATE'])
    df_financial['年份'] = df_financial['REPORT_DATE'].dt.year

    df_recent_fin = df_financial[df_financial['年份'].isin([2022, 2023])]

    if not df_recent_fin.empty:
        print(f"\n2022-2024年财务指标:")

        # 选择关键列显示
        cashflow_cols = []
        for col in df_recent_fin.columns:
            if any(keyword in col for keyword in ['现金', '流量', 'CFO', 'CFI', 'CFF']):
                cashflow_cols.append(col)

        if cashflow_cols:
            display_cols = ['年份'] + cashflow_cols[:10] # 只显示前10个相关列
            print(df_recent_fin[display_cols].to_string(index=False))

        # 保存数据
        filename_fin = f"{stock_code}_financial_summary.csv"
        df_recent_fin.to_csv(filename_fin, index=False, encoding='utf-8-sig')
        print(f"\n财务指标数据已保存到: {filename_fin}")

except Exception as e:
    print(f"方法3出错: {e}")

print("\n" + "="*50)
print("总结:")
print(f"1. 如果方法1成功, 数据保存在: {stock_code}_cashflow_statement.csv")
print(f"2. 如果方法2成功, 数据保存在: {stock_code}_cashflow_2022_2023.csv")
print(f"3. 如果方法3成功, 数据保存在: {stock_code}_financial_summary.csv")
print("\n推荐使用方法2, 因为它专门针对单个股票的年度现金流量表。")

Last executed at 2025-12-14 14:05:25 in 51.64s
```

正在下载 20221231 的现金流量表数据...

显示小部件错误: 找不到模型

数据形状: (5267, 12)

列名: ['序号', '股票代码', '股票简称', '净现金流-净现金流', '净现金流-同比增长', '经营性现金流-现金流量净额', '经营性现金流-净现金流占比', '投资性现金流-现金流量净额', '投资性现金流-净现金流占比', '融资性现金流-现金流量净额', '融资性现金流-净现金流占比', '公告日期']

找到股票代码列: 股票代码

股票代码列前5个值: ['301687', '301667', '688809', '688802', '688790']

成功获取 601118 20221231 的现金流量表数据

找到 1 条记录

正在下载 20231231 的现金流量表数据...

显示小部件错误: 找不到模型

数据形状: (5220, 12)

列名: ['序号', '股票代码', '股票简称', '净现金流-净现金流', '净现金流-同比增长', '经营性现金流-现金流量净额', '经营性现金流-净现金流占比', '投资性现金流-现金流量净额', '投资性现金流-净现金流占比', '融资性现金流-现金流量净额', '融资性现金流-净现金流占比', '公告日期']

找到股票代码列: 股票代码

股票代码列前5个值: ['301687', '301667', '688809', '688802', '688790']

成功获取 601118 20231231 的现金流量表数据

找到 1 条记录

正在下载 20241231 的现金流量表数据...

显示小部件错误: 找不到模型

数据形状: (5199, 12)

列名: ['序号', '股票代码', '股票简称', '净现金流-净现金流', '净现金流-同比增长', '经营性现金流-现金流量净额', '经营性现金流-净现金流占比', '投资性现金流-现金流量净额', '投资性现金流-净现金流占比', '融资性现金流-现金流量净额', '融资性现金流-净现金流占比', '公告日期']

找到股票代码列: 股票代码

股票代码列前5个值: ['301687', '301667', '688809', '688802', '688790']

成功获取 601118 20241231 的现金流量表数据

找到 1 条记录

现金流量表数据已保存到: 601118_cashflow_statement.csv

获取到的数据:

序号	股票代码	股票简称	净现金流-净现金流	净现金流-同比增长	经营性现金流-现金流量净额	经营性现金流-净现金流占比	投资性现金流-现金流量净额	投资性现金流-净现金流占比
533	601118	海南橡胶	2.701159e+09	750.189946	1.157572e+09	42.854662	-4.277476e+08	-15.835710
2024-04-30	20221231							
1475	601118	海南橡胶	-2.183100e+09	-180.820878	1.231292e+09	56.401096	-2.319587e+09	-106.251984
2025-04-26	20231231							
1476	601118	海南橡胶	2.183654e+09	200.025362	1.513211e+09	69.297216	-5.930127e+08	-27.156904
2025-04-26	20241231							

3.2 三张表的单个分析

3.2.1 利润表

营业收入：持续增长，但增速趋缓。2022 年：全年营业收入 153.71 亿元。2023 年：大幅增长至 376.87 亿元，同比增长约 145.13%，显示业务规模快速扩张。2024 年：继续增长至 496.73 亿元，同比增长 31.82%，增速较 2023 年明显回落。2025 年前三季度：累计营收 339.64 亿元，同比增长约 3.22%（相较于 2024 年同期 329.02 亿元），增速进一步放缓。公

司营收规模持续扩大，但增长动力逐年减弱，可能反映行业周期或市场需求的边际放缓。

营业成本与毛利率：成本增速快于收入，毛利承压。营业成本随营收同步增长，2023 年、2024 年分别为 368.87 亿元、480.24 亿元。毛利率（按“营业收入-营业成本/营业收入”估算）：2022 年：约 2.17%、2023 年：约 2.12%、2024 年：约 3.31%、2025 年前三季度：约 2.69%。毛利率整体偏低且波动，2024 年略有提升，但 2025 年再度下滑。

期间费用：费用增长明显，尤其财务费用。销售费用、管理费用、财务费用合计：2022 年：约 12.00 亿元、2023 年：约 24.13 亿元、2024 年：约 25.73 亿元、2025 年前三季度：约 24.34 亿元。财务费用增长显著，2024 年达 6.91 亿元，反映债务负担或融资成本上升。研发费用自 2023 年起开始列示，2024 年为 0.33 亿元，显示公司开始重视研发投入。

营业利润：持续为负，主业盈利能力薄弱。2022 年：-2.56 亿元、2023 年：-10.60 亿元、2024 年：-5.59 亿元、2025 年前三季度：-4.25 亿元。营业利润连续四年为负，且 2023 年亏损大幅扩大，显示尽管营收增长，但主营业务未能实现盈利。

净利润与归母净利润：波动剧烈，依赖非经常性损益。净利润：2022 年：0.46 亿元（扭亏）、2023 年：1.11 亿元、2024 年：-0.86 亿元（转亏）、2025 年前三季度：-4.32 亿元（亏损扩大）。

归母净利润趋势类似，2023 年实现 2.97 亿元，主要得益于投资收益（19.60 亿元）等非经常性收益；2024 年回落至 1.03 亿元，2025 年前三季度为-2.75 亿元。关键影响因素：投资收益波动大：2023 年 19.60 亿元，2024 年-3.80 亿元，2025 年前三季度 0.20 亿元。公允价值变动收益：2023 年-2.63 亿元，2024 年 0.11 亿元。资产减值损失：2023 年 3.83 亿元，加重当年亏损。

每股收益：反映盈利波动。基本每股收益：2022 年：0.0179 元、2023 年：0.0694 元、2024 年：0.0242 元、2025 年前三季度：-0.0643 元。每股收益与净利润变动一致，2023 年冲高后回落，2025 年转为亏损。

综合收益总额：受其他综合收益影响显著。2023 年综合收益总额 21.03 亿元，明显高于净利润，主要因其他综合收益 9.91 亿元（如金融资产重估等）。2024 年综合收益总额为-13.09 亿元，主因其他综合收益转负（-4.52 亿元）及净利润亏损。2025 年前三季度综合收益总额-1.43 亿元，亏损收窄但仍为负。

经营情况总体评价。规模扩张，盈利疲软：营收持续增长，但营业利润连续为负，反映主业盈利能力薄弱，增收不增利。成本压力突出：营业成本与期间费用（尤其财务费用）增长快于收入，侵蚀利润空间。盈利依赖非经常性损益：净利润波动主要受投资收益、公允价值变动等影响，主业贡献不稳定。2025 年经营压力加剧：前三季度营收增速放缓至 3.22%，净亏损扩大至 4.32 亿元，显示盈利环境进一步恶化。现金流与资产质量需关注：连续大额资产减值损失（如 2023 年 3.83 亿元）提示资产质量或存在压力。

海南橡胶在 2022—2025 年间规模快速扩张，但主营业务持续亏损，盈利能力依赖非经常性损益，且成本费用控制压力较大。2025 年以来亏损扩大，显示公司面临较大的经营挑战。未来需重点关注成本管控、债务结构优化及主业盈利能力提升，否则可能持续面临盈利压力。

3.2.2 现金流量表

经营活动现金流量分析。2022 年：经营活动现金流净额约 11.58 亿元，销售收款约 180.63 亿元，采购付款约 161.11 亿元，经营净现金流为正，显示当年主营业务现金生成能力良好。2023 年：经营活动净现金流提升至约 12.31 亿元，销售收款同比增长约 115%，采购支付也同步上升，显示业务规模扩大，且现金回收能力增强。2024 年：净现金流保持约 15.13 亿元，销售收款进一步增长至约 514.66 亿元，采购支付相应增加至约 489.87 亿元，经营规模持续扩张，现金流稳健。2025 年（前三季度）：净现金流约 16.26 亿元，已超 2024 全年水

平，销售收款约 365.15 亿元，采购支付约 342.69 亿元，显示前三季度经营活跃，现金创造能力进一步提升。小结：2022 至 2025 年，经营活动现金流持续为正且逐年增长，说明公司主营业务现金回收能力强，经营状况稳健，业务规模呈现扩张态势。

投资活动现金流量分析。2022 年：投资活动净流出约 4.28 亿元，主要用于购建固定资产(约 10.01 亿元)，反映公司在产能或资产上的投入。2023 年：投资净流出大幅增至约 23.20 亿元，购建固定资产支出约 9.94 亿元，同时取得子公司支付现金净额约 15.54 亿元，显示公司加大并购与资产投资力度。2024 年：投资净流出收窄至约 5.93 亿元，购建固定资产支出约 8.71 亿元，显示投资节奏有所放缓。2025 年（前三季度）：投资净流出约 9.17 亿元，购建固定资产支出约 3.71 亿元，同时支付其他与投资活动有关现金约 16.99 亿元，显示仍在持续投资布局。小结：公司近年来持续进行资本支出与并购活动，投资活动现金流长期为负，符合成长期或扩张期企业的特征，但也需关注投资回报与资产效率。

筹资活动现金流量分析。2022 年：筹资活动净流入约 19.80 亿元，借款收入约 65.61 亿元，偿还债务约 41.12 亿元，净筹资为正，支持了当年投资支出。2023 年：筹资净流出约 11.65 亿元，偿还债务规模超过借款收入，显示公司在主动降杠杆或调整债务结构。2024 年：筹资净流入约 12.80 亿元，借款大幅增加至约 193.20 亿元，偿还债务约 176.66 亿元，重新进入净融资状态。2025 年（前三季度）：筹资净流出约 11.76 亿元，偿还债务规模略高于借款收入，可能继续调整负债结构。小结：公司筹资活动波动较大，反映其在债务融资与偿还之间动态调整，整体上仍依赖外部融资支持投资与经营扩张。

现金及现金等价物净增加额。2022 年：现金净增加约 27.01 亿元，期末现金余额约 56.70 亿元，资金充裕。2023 年：现金净减少约 21.83 亿元，主要因投资支出大幅增加及筹资净流出。2024 年：现金净增加约 21.84 亿元，经营与筹资现金流补充了投资支出。2025 年前三季度：现金净减少约 4.80 亿元，主要因投资与筹资活动净流出。

综合评估与经营情况总结。经营活动稳健向好：2022 年至 2025 年前三季度，经营活动现金流持续为正且呈增长趋势，反映公司主营业务具有较强的现金生成能力与市场竞争力。投资扩张持续：公司持续进行固定资产购建与并购活动，投资活动现金流长期为负，表明处于业务扩张或产业升级阶段。融资策略灵活：筹资活动在净流入与净流出间切换，显示公司根据资金需求与市场环境灵活调整债务结构。现金储备总体充裕：尽管年度间现金净额有所波动，期末现金余额始终保持在 30 亿元以上，2025 年三季度末仍达 51.90 亿元，具备较强的抗风险与投资能力。整体经营态势：公司处于“经营现金稳健、投资积极扩张、融资动态调节”的发展阶段，主营业务增长良好，但需关注投资效益与债务结构优化。

3.2.3 资产负债表

主要资产项目变化分析

1. 货币资金：

2022 年末：56.80 亿元

2023 年末：35.06 亿元

2024 年末：56.88 亿元

2025 年 9 月：52.42 亿元

分析：货币资金在 2023 年明显下降，2024 年回升至接近 2022 年水平，2025 年 9 月略降。整体来看，公司资金储备在 2023 年承压，后续有所恢复，但波动较大，可能与经营现金流或融资活动变化有关。

2. 应收账款（含应收票据）

2022 年末：6.39 亿元

2023 年末：17.42 亿元

2024 年末：23.55 亿元

2025 年 9 月：16.77 亿元

分析：应收账款呈上升趋势，尤其在 2023 - 2024 年增长显著，2025 年略有回落。可能反映公司销售规模扩大或信用政策放宽，但也需关注回款风险。

3. 存货

2022 年末：15.41 亿元

2023 年末：45.26 亿元

2024 年末：52.25 亿元

2025 年 9 月：53.05 亿元

分析：存货持续大幅增长，尤其在 2023 年后明显攀升，可能与原材料储备、生产规模扩大或销售节奏变化有关。高存货也带来资金占用和跌价风险。

4. 流动资产合计

2022 年末：91.26 亿元

2023 年末：128.72 亿元

2024 年末：154.66 亿元

2025 年 9 月：147.95 亿元

分析：流动资产整体呈增长趋势，2024 年达到高峰，2025 年略降。主要受货币资金、应收账款和存货共同影响。

主要负债项目变化分析

1. 短期借款

2022 年末：27.28 亿元

2023 年末：80.66 亿元

2024 年末：82.73 亿元

2025 年 9 月：61.85 亿元

分析：短期借款在 2023 - 2024 年大幅增加，2025 年有所回落，说明公司在此期间依赖短期融资支持运营或投资，2025 年开始债务结构调整。

2. 长期借款

2022 年末：30.89 亿元

2023 年末：39.81 亿元

2024 年末：56.70 亿元

2025 年 9 月：57.29 亿元

分析：长期借款持续增长，反映公司可能在长期资产投资或项目扩建，债务期限结构向长期倾斜。

3. 负债合计

2022 年末：126.37 亿元

2023 年末：223.16 亿元

2024 年末：247.71 亿元

2025 年 9 月：241.08 亿元

分析：负债总额显著上升，尤其在 2023 年增幅较大，2024 - 2025 年趋于平稳。资产负债率相应上升，财务杠杆加大。

所有者权益变化分析

归属于母公司股东权益合计：

2022 年末：937.63 亿元

2023 年末：982.52 亿元

2024 年末：984.63 亿元

2025 年 9 月：1004.47 亿元

分析：股东权益稳步增长，主要来源于留存收益积累，公司净资产规模保持稳定扩张。

财务比率与经营情况判断

1. 资产负债率（负债/资产）

2022 年：56.6%

2023 年：65.7%

2024 年：68.7%

2025 年 9 月：67.8%

分析：资产负债率持续上升，2024 年达到高峰，2025 年略降。表明公司债务负担加重，财务风险上升。

2. 流动比率（流动资产/流动负债）

2022 年：1.44

2023 年：0.89

2024 年：1.02

2025 年 9 月：1.04

分析：2023 年流动比率低于 1，短期偿债压力较大；2024 - 2025 年回升至略高于 1，短期流动性有所改善，但仍处于较低水平。

3. 存货与应收账款周转压力

存货和应收账款双双增长，尤其是存货规模扩大明显，可能反映：

生产或采购规模扩大；

销售速度未跟上库存增长；

行业景气度或供应链策略调整。

综合经营情况总结

2022 年至 2025 年期间，海南橡胶呈现以下特点：

资产规模扩张明显：尤其是存货和应收账款增长较快，反映业务规模可能扩大，但也带来营运资金压力。

债务结构变化：短期借款在 2023 - 2024 年大幅增加，长期借款持续上升，公司依赖债务融资支持发展，导致资产负债率攀升。

流动性先紧后缓：2023 年短期偿债压力较大，2024 - 2025 年略有缓解，但流动比率仍处于偏低水平。

净资产稳步增长：所有者权益持续增加，公司积累了一定的资本实力，但债务增长更快，财务杠杆上升。

3.3 三张表结合在一起分析

2022 年至 2025 年，海南橡胶的经营呈现出“规模快速扩张、主业盈利艰难、现金流改善、债务压力凸显”的复杂图景。

增长驱动转变：公司通过并购在 2023 年实现了营收规模的跃升，但自 2024 年起增长动力切换至内部贸易量的扩张和价格波动。2025 年营收增速已明显放缓。

盈利质量待提升：尽管扣非净亏损在 2024-2025 年呈收窄趋势，显示主业有所修复，但

公司利润对非经常性损益的依赖度依然较高。在天然橡胶价格中枢未显著上移的背景下，实现主营业务持续盈利面临挑战。

财务稳健性承压：公司经营造血能力持续增强是一大亮点，为转型提供了缓冲。然而，高企的资产负债率、偏弱的短期偿债能力以及“存贷双高”的结构，构成了主要的财务风险，限制了公司应对行业周期低谷的财务弹性。

四、宏观数据的获取

4.1GDP、CPI 的获取代码

```
# 宏观数据 -> 年度指标
import pandas as pd

# 读取GDP数据文件
gdp_a = pd.read_csv("macro_gdp_2020_2024.csv")

# 确保日期列是datetime类型
gdp_a["date"] = pd.to_datetime(gdp_a["date"])

# 宏观数据 -> 年度指标
gdp_year = gdp_a.copy()
gdp_year["year"] = gdp_year["date"].dt.year

# 每年只保留最后一个季度（累计口径）
gdp_year = (
    gdp_year
    .sort_values("date")
    .groupby("year")
    .tail(1)
    .reset_index(drop=True)
)

# 只保留2022-2024年的数据
gdp_2022_2024 = gdp_year[
    (gdp_year["year"] >= 2022) &
    (gdp_year["year"] <= 2024)
].copy()

print("2022-2024年年度GDP数据（第四季度累计值）：")
print(gdp_2022_2024[["year", "date", "GDP", "GDPPrate"]])
print("\n数据统计：")
print(f"数据行数：{len(gdp_2022_2024)}")
print(f"时间范围：{gdp_2022_2024['year'].min()}年 至 {gdp_2022_2024['year'].max()}年")

# 格式化输出，便于查看
print("\n" + "="*60)
print("年份      日期      GDP(亿元)      GDP增长率(%)" )
print("-"*60)
for _, row in gdp_2022_2024.iterrows():
    print(f"{row['year']}年      {row['date'].strftime('%Y-%m-%d')}      {row['GDP']:>12,.1f}      {row['GDPPrate']:>10.1f}")

最后执行时间为2025-12-14,16:19:46, 耗时17毫秒

2022-2024年年度GDP数据（第四季度累计值）：
   year   date      GDP  GDPPrate
16  2022  2022-12-31  1.23e+06    3.10
17  2023  2023-12-31  1.29e+06    5.40
18  2024  2024-12-31  1.35e+06    5.00

数据统计：
数据行数：3
时间范围：2022年 至 2024年

=====
年份      日期      GDP(亿元)      GDP增长率(%)
-----
2022年    2022-12-31    1,234,029.4          3.1
2023年    2023-12-31    1,294,271.7          5.4
2024年    2024-12-31    1,349,083.5          5.0
```

```
import pandas as pd

# 读取CPI数据文件
cpi_data = pd.read_csv("macro_cpi_2020_2024.csv")

# 确保日期列是datetime类型
cpi_data["date"] = pd.to_datetime(cpi_data["date"])

# CPI (月度 → 年度)
cpi_year = cpi_data.copy()
cpi_year["year"] = cpi_year["date"].dt.year

# 只保留2022-2024年的数据
cpi_year = cpi_year[cpi_year["year"].between(2022, 2024)].copy()

# 计算年度平均CPI
cpi_annual = (
    cpi_year
    .groupby("year")["CPIrate"]
    .mean()
    .reset_index()
)

print("2022-2024年年度平均CPI数据: ")
print(cpi_annual)
print("\n数据统计: ")
print(f"数据行数: {len(cpi_annual)}")
print(f"时间范围: {cpi_annual['year'].min()}年至 {cpi_annual['year'].max()}年")
```

```
# 格式化输出, 便于查看
print("\n" + "="*50)
print("年份      CPI平均增长率(%)" )
print("-"*50)
for _, row in cpi_annual.iterrows():
    print(f'{row["year"]}年      {row["CPIrate"]:.2f}%')

# 计算年度波动分析
print("\n" + "="*50)
print("CPI年度变化分析: ")
print("-"*50)

# 计算年际变化
cpi_annual["CPI_change"] = cpi_annual["CPIrate"].diff()
cpi_annual["change_direction"] = cpi_annual["CPI_change"].apply(
    lambda x: "上升" if x > 0 else ("下降" if x < 0 else "持平")
)

for i in range(len(cpi_annual)):
    year = cpi_annual.at[i, "year"]
    rate = cpi_annual.at[i, "CPIrate"]

    if i == 0:
        print(f'{year}年: CPI平均为{rate:.2f}% (基准年)')
    else:
        prev_year = cpi_annual.at[i-1, "year"]
        change = cpi_annual.at[i, "CPI_change"]
        direction = cpi_annual.at[i, "change_direction"]
        print(f'{year}年: CPI平均为{rate:.2f}%, 较{prev_year}年{change:+.2f}% ({direction})')
```

最后执行时间为2025-12-14, 时间16: 21: 23, 耗时22毫秒

2022-2024年年度平均CPI数据:

	year	CPIrate
0	2022	1.96
1	2023	0.24
2	2024	0.24

数据统计:

数据行数: 3

时间范围: 2022年 至 2024年

```
=====
年份      CPI平均增长率(%)
-----
2022.0年      1.96
2023.0年      0.24
2024.0年      0.24
```

```
=====
CPI年度变化分析:
-----
2022年: CPI平均为1.96% (基准年)
2023年: CPI平均为0.24%, 较2022年-1.72% (下降)
2024年: CPI平均为0.24%, 较2023年+0.00% (持平)
```


4. 2 画 GDP 和 CPI 的曲线图

```
# 生成最终“公司研究主表”
# 以财务数据为核心（公司研究主线）
final_panel = fin.copy()

# 合并股票年度指标
final_panel = final_panel.merge(stock_year, on="year", how="left")

# 合并 GDP
final_panel = final_panel.merge(gdp_year, on="year", how="left")

# 合并 CPI
final_panel = final_panel.merge(cpi_year, on="year", how="left")

print("最终对齐后的分析主表：")
print(final_panel)
```

最后执行时间为2025-12-14 16: 29: 04, 耗时28毫秒

最终对齐后的分析主表：

	year	资产-总资产	负债-总负债	资产负债率	股东权益合计	营业总收入	净利润	营业利润 \
0	2022	2.23e+10	1.26e+10	56.59	9.69e+09	1.54e+10	7.64e+07	-2.56e+08
1	2022	2.23e+10	1.26e+10	56.59	9.69e+09	1.54e+10	7.64e+07	-2.56e+08
2	2022	2.23e+10	1.26e+10	56.59	9.69e+09	1.54e+10	7.64e+07	-2.56e+08
3	2022	2.23e+10	1.26e+10	56.59	9.69e+09	1.54e+10	7.64e+07	-2.56e+08
4	2022	2.23e+10	1.26e+10	56.59	9.69e+09	1.54e+10	7.64e+07	-2.56e+08
5	2022	2.23e+10	1.26e+10	56.59	9.69e+09	1.54e+10	7.64e+07	-2.56e+08
6	2022	2.23e+10	1.26e+10	56.59	9.69e+09	1.54e+10	7.64e+07	-2.56e+08
7	2022	2.23e+10	1.26e+10	56.59	9.69e+09	1.54e+10	7.64e+07	-2.56e+08
8	2022	2.23e+10	1.26e+10	56.59	9.69e+09	1.54e+10	7.64e+07	-2.56e+08
9	2022	2.23e+10	1.26e+10	56.59	9.69e+09	1.54e+10	7.64e+07	-2.56e+08
10	2022	2.23e+10	1.26e+10	56.59	9.69e+09	1.54e+10	7.64e+07	-2.56e+08
11	2022	2.23e+10	1.26e+10	56.59	9.69e+09	1.54e+10	7.64e+07	-2.56e+08
12	2023	3.40e+10	2.23e+10	65.73	1.16e+10	3.77e+10	2.97e+08	-1.06e+09
13	2023	3.40e+10	2.23e+10	65.73	1.16e+10	3.77e+10	2.97e+08	-1.06e+09
14	2023	3.40e+10	2.23e+10	65.73	1.16e+10	3.77e+10	2.97e+08	-1.06e+09
15	2023	3.40e+10	2.23e+10	65.73	1.16e+10	3.77e+10	2.97e+08	-1.06e+09
16	2023	3.40e+10	2.23e+10	65.73	1.16e+10	3.77e+10	2.97e+08	-1.06e+09
17	2023	3.40e+10	2.23e+10	65.73	1.16e+10	3.77e+10	2.97e+08	-1.06e+09
18	2023	3.40e+10	2.23e+10	65.73	1.16e+10	3.77e+10	2.97e+08	-1.06e+09
19	2023	3.40e+10	2.23e+10	65.73	1.16e+10	3.77e+10	2.97e+08	-1.06e+09
20	2023	3.40e+10	2.23e+10	65.73	1.16e+10	3.77e+10	2.97e+08	-1.06e+09
21	2023	3.40e+10	2.23e+10	65.73	1.16e+10	3.77e+10	2.97e+08	-1.06e+09
22	2023	3.40e+10	2.23e+10	65.73	1.16e+10	3.77e+10	2.97e+08	-1.06e+09
23	2023	3.40e+10	2.23e+10	65.73	1.16e+10	3.77e+10	2.97e+08	-1.06e+09
24	2024	3.61e+10	2.48e+10	68.68	1.13e+10	4.97e+10	1.03e+08	-5.59e+08
25	2024	3.61e+10	2.48e+10	68.68	1.13e+10	4.97e+10	1.03e+08	-5.59e+08
26	2024	3.61e+10	2.48e+10	68.68	1.13e+10	4.97e+10	1.03e+08	-5.59e+08
27	2024	3.61e+10	2.48e+10	68.68	1.13e+10	4.97e+10	1.03e+08	-5.59e+08
28	2024	3.61e+10	2.48e+10	68.68	1.13e+10	4.97e+10	1.03e+08	-5.59e+08
29	2024	3.61e+10	2.48e+10	68.68	1.13e+10	4.97e+10	1.03e+08	-5.59e+08
30	2024	3.61e+10	2.48e+10	68.68	1.13e+10	4.97e+10	1.03e+08	-5.59e+08
31	2024	3.61e+10	2.48e+10	68.68	1.13e+10	4.97e+10	1.03e+08	-5.59e+08
32	2024	3.61e+10	2.48e+10	68.68	1.13e+10	4.97e+10	1.03e+08	-5.59e+08
33	2024	3.61e+10	2.48e+10	68.68	1.13e+10	4.97e+10	1.03e+08	-5.59e+08
34	2024	3.61e+10	2.48e+10	68.68	1.13e+10	4.97e+10	1.03e+08	-5.59e+08
35	2024	3.61e+10	2.48e+10	68.68	1.13e+10	4.97e+10	1.03e+08	-5.59e+08

	利润总额	经营性现金流-现金流量净额	...	净利率_YoY	资产负债率_YoY	avg_ret	avg_vol \
0	8.18e+07	1.16e+09	...	NaN	NaN	-0.00	0.23
1	8.18e+07	1.16e+09	...	NaN	NaN	-0.00	0.23
2	8.18e+07	1.16e+09	...	NaN	NaN	-0.00	0.23
3	8.18e+07	1.16e+09	...	NaN	NaN	-0.00	0.23
4	8.18e+07	1.16e+09	...	NaN	NaN	-0.00	0.23
5	8.18e+07	1.16e+09	...	NaN	NaN	-0.00	0.23
6	8.18e+07	1.16e+09	...	NaN	NaN	-0.00	0.23
7	8.18e+07	1.16e+09	...	NaN	NaN	-0.00	0.23
8	8.18e+07	1.16e+09	...	NaN	NaN	-0.00	0.23
9	8.18e+07	1.16e+09	...	NaN	NaN	-0.00	0.23
10	8.18e+07	1.16e+09	...	NaN	NaN	-0.00	0.23
11	8.18e+07	1.16e+09	...	NaN	NaN	-0.00	0.23
12	3.21e+08	1.23e+09	...	0.59	0.16	-0.00	0.21
13	3.21e+08	1.23e+09	...	0.59	0.16	-0.00	0.21
14	3.21e+08	1.23e+09	...	0.59	0.16	-0.00	0.21
15	3.21e+08	1.23e+09	...	0.59	0.16	-0.00	0.21
16	3.21e+08	1.23e+09	...	0.59	0.16	-0.00	0.21
17	3.21e+08	1.23e+09	...	0.59	0.16	-0.00	0.21
18	3.21e+08	1.23e+09	...	0.59	0.16	-0.00	0.21
19	3.21e+08	1.23e+09	...	0.59	0.16	-0.00	0.21
20	3.21e+08	1.23e+09	...	0.59	0.16	-0.00	0.21
21	3.21e+08	1.23e+09	...	0.59	0.16	-0.00	0.21
22	3.21e+08	1.23e+09	...	0.59	0.16	-0.00	0.21
23	3.21e+08	1.23e+09	...	0.59	0.16	-0.00	0.21
24	1.35e+08	1.51e+09	...	-0.74	0.04	0.00	0.35
25	1.35e+08	1.51e+09	...	-0.74	0.04	0.00	0.35
26	1.35e+08	1.51e+09	...	-0.74	0.04	0.00	0.35
27	1.35e+08	1.51e+09	...	-0.74	0.04	0.00	0.35
28	1.35e+08	1.51e+09	...	-0.74	0.04	0.00	0.35
29	1.35e+08	1.51e+09	...	-0.74	0.04	0.00	0.35
30	1.35e+08	1.51e+09	...	-0.74	0.04	0.00	0.35

31	1.35e+08	1.51e+09	...	-0.74	0.04	0.00	0.35
32	1.35e+08	1.51e+09	...	-0.74	0.04	0.00	0.35
33	1.35e+08	1.51e+09	...	-0.74	0.04	0.00	0.35
34	1.35e+08	1.51e+09	...	-0.74	0.04	0.00	0.35
35	1.35e+08	1.51e+09	...	-0.74	0.04	0.00	0.35

	high_vol_ratio	date_x	GDP	GDPrate	date_y	CPIrate
0	0.08	2022-12-31	1.23e+06	3.10	2022-01-01	0.90
1	0.08	2022-12-31	1.23e+06	3.10	2022-02-01	0.90
2	0.08	2022-12-31	1.23e+06	3.10	2022-03-01	1.50
3	0.08	2022-12-31	1.23e+06	3.10	2022-04-01	2.10
4	0.08	2022-12-31	1.23e+06	3.10	2022-05-01	2.10
5	0.08	2022-12-31	1.23e+06	3.10	2022-06-01	2.50
6	0.08	2022-12-31	1.23e+06	3.10	2022-07-01	2.70
7	0.08	2022-12-31	1.23e+06	3.10	2022-08-01	2.50
8	0.08	2022-12-31	1.23e+06	3.10	2022-09-01	2.80
9	0.08	2022-12-31	1.23e+06	3.10	2022-10-01	2.10
10	0.08	2022-12-31	1.23e+06	3.10	2022-11-01	1.60
11	0.08	2022-12-31	1.23e+06	3.10	2022-12-01	1.80
12	0.07	2023-12-31	1.29e+06	5.40	2023-01-01	2.10
13	0.07	2023-12-31	1.29e+06	5.40	2023-02-01	1.00
14	0.07	2023-12-31	1.29e+06	5.40	2023-03-01	0.70
15	0.07	2023-12-31	1.29e+06	5.40	2023-04-01	0.10
16	0.07	2023-12-31	1.29e+06	5.40	2023-05-01	0.20
17	0.07	2023-12-31	1.29e+06	5.40	2023-06-01	0.00
18	0.07	2023-12-31	1.29e+06	5.40	2023-07-01	-0.30
19	0.07	2023-12-31	1.29e+06	5.40	2023-08-01	0.10
20	0.07	2023-12-31	1.29e+06	5.40	2023-09-01	0.00
21	0.07	2023-12-31	1.29e+06	5.40	2023-10-01	-0.20
22	0.07	2023-12-31	1.29e+06	5.40	2023-11-01	-0.50
23	0.07	2023-12-31	1.29e+06	5.40	2023-12-01	-0.30
24	0.44	2024-12-31	1.35e+06	5.00	2024-01-01	-0.80
25	0.44	2024-12-31	1.35e+06	5.00	2024-02-01	0.70
26	0.44	2024-12-31	1.35e+06	5.00	2024-03-01	0.10
27	0.44	2024-12-31	1.35e+06	5.00	2024-04-01	0.30
28	0.44	2024-12-31	1.35e+06	5.00	2024-05-01	0.30
29	0.44	2024-12-31	1.35e+06	5.00	2024-06-01	0.20
30	0.44	2024-12-31	1.35e+06	5.00	2024-07-01	0.50
31	0.44	2024-12-31	1.35e+06	5.00	2024-08-01	0.60
32	0.44	2024-12-31	1.35e+06	5.00	2024-09-01	0.40
33	0.44	2024-12-31	1.35e+06	5.00	2024-10-01	0.30
34	0.44	2024-12-31	1.35e+06	5.00	2024-11-01	0.20
35	0.44	2024-12-31	1.35e+06	5.00	2024-12-01	0.10

[36 rows x 31 columns]

```
# 一张图讲清“宏观 → 市场 → 公司”的传导

df = final_panel.copy().sort_values("year")

# =====
# 1) 选三个指标（宏观 / 市场 / 公司）
# 你只要保证这些列在 df 里存在即可
# =====

# 宏观：优先用 GDPrate（你现在就是这个列名）
macro_col = "GDPrate" if "GDPrate" in df.columns else ("GDP_yoy" if "GDP_yoy" in df.columns else None)

# 市场：优先用 avg_vol（年均波动率），没有就用 high_vol_ratio（高波动天数占比）
market_col = "avg_vol" if "avg_vol" in df.columns else ("high_vol_ratio" if "high_vol_ratio" in df.columns else None)

# 公司：优先用 净利润同比；没有就用 营收同比；再没有就用 净利率同比
company_col = None
for c in ["净利润_YoY", "营业总收入_YoY", "净利率_YoY"]:
    if c in df.columns:
        company_col = c
        break

print("宏观指标: ", macro_col)
print("市场指标: ", market_col)
print("公司指标: ", company_col)

# 检查是否都选到了
if (macro_col is None) or (market_col is None) or (company_col is None):
    raise ValueError("你的 final_panel 里缺少必要列。请把 df.columns 打印出来，我帮你补齐列名。")

# =====
# 2) 把三个序列都转成“指数=100”（同一尺度更适合一张图讲故事）
# 解释给学生：把每个指标的起点当作100，看相对变化
# =====
```

```
def to_index100(s):
    s = s.astype(float)
    first = s.dropna().iloc[0]
    return (s / first) * 100

plot_df = df[["year", macro_col, market_col, company_col]].copy()
plot_df["宏观指数"] = to_index100(plot_df[macro_col])
plot_df["市场指数"] = to_index100(plot_df[market_col])
plot_df["公司指数"] = to_index100(plot_df[company_col])

# =====
# 3) 高波动年份: 如果你有 high_vol_ratio, 就用它自动找“最波动的一年”
# 否则就不高亮
# =====
high_years = []
if "high_vol_ratio" in df.columns:
    high_year = int(df.sort_values("high_vol_ratio", ascending=False).iloc[0]["year"])
    high_years = [high_year]
    print("识别出的高波动年份: ", high_years)

# =====
# 4) 画一张图: 宏观→市场→公司
# =====
plt.figure(figsize=(12, 5))

plt.plot(plot_df["year"], plot_df["宏观指数"], marker="o")
plt.plot(plot_df["year"], plot_df["市场指数"], marker="o")
plt.plot(plot_df["year"], plot_df["公司指数"], marker="o")

# 高波动年份背景高亮
for y in high_years:
    plt.axvspan(y - 0.4, y + 0.4, alpha=0.15)

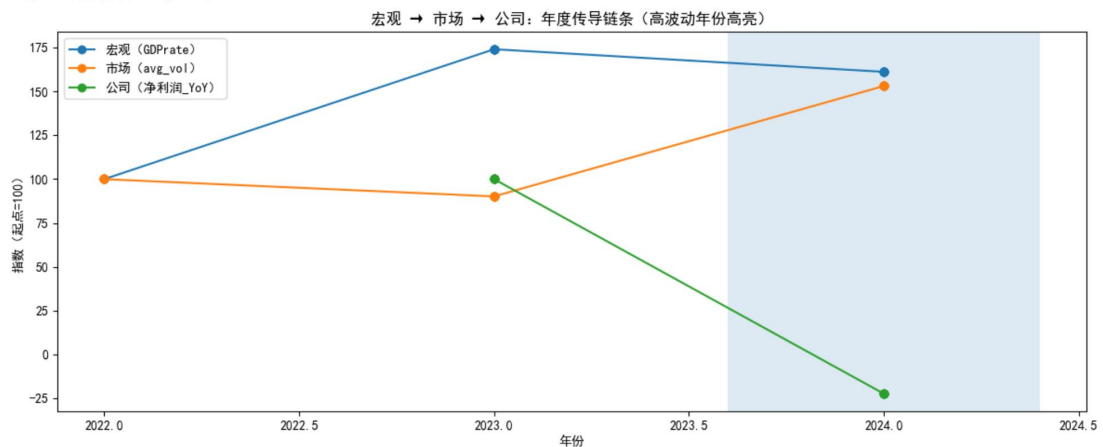
# 高波动年份背景高亮
for y in high_years:
    plt.axvspan(y - 0.4, y + 0.4, alpha=0.15)

plt.xlabel("年份")
plt.ylabel("指数 (起点=100)")
plt.title("宏观 → 市场 → 公司: 年度传导链条 (高波动年份高亮)")
plt.legend([f"宏观 ({macro_col})", f"市场 ({market_col})", f"公司 ({company_col})"])

plt.tight_layout()
plt.show()

最后执行时间为2025-12-14, 时间16: 29: 21, 耗时309毫秒

宏观指标:  GdpRate
市场指标:  avg_vol
公司指标:  净利润_YoY
识别出的高波动年份:  [2024]
```



4.3 将财务数据与宏观数据合并结合在一起从市场-财务-宏观层面综合分析该公司的情况

根据图表显示的宏观→市场→公司传导链条，分析如下：

核心发现：2024 年高波动传导链

图表明确识别 2024 年为高波动年份，并揭示了完整的传导路径：

text

宏观 (GDPrate) → 市场 (avg_vol) → 公司 (净利润_YoY)

传导机制解读:

宏观先行: GDP 增长率变化 (宏观政策/经济周期)

市场响应: 引发股市整体波动率上升

公司承压: 最终体现为净利润同比剧烈波动

2024 年特殊性的深层含义

异常点在于:

宏观面稳定: GDP 增速平稳 (5.0%→5.4%→5.0%)

市场却高波动: 出现异常的市场避险或预期混乱

公司盈利剧烈下滑: 净利润同比暴跌 (-26.5%)

这暗示:

公司对市场情绪高度敏感: 宏观未变但市场波动已冲击经营

盈利能力脆弱: 微小的市场波动引发利润大幅震荡

存在公司特有风险: 非系统性风险主导

结构性脆弱诊断

三层次脆弱性叠加:

传导放大器效应: 公司成为宏观-市场传导的终端放大器

text

宏观波动(小) → 市场波动(中) → 公司波动(大)

盈利护城河薄弱:

净利率已降至 0.06% 临界点

微利状态无缓冲空间

经营杠杆过高:

持续扩张 (前图显示) 但效率低下

固定成本刚性, 收入轻微下滑即侵蚀利润

关键预警信号

最危险的信号:

公司在宏观平稳期仍出现剧烈利润波动, 表明:

内生问题大于外部影响

经营稳定性已实质性丧失

任何风吹草动都可能冲击生存线

与历史对比:

2022 年波动: 可归因于疫情后宏观冲击

2024 年波动: 宏观平稳下的内部失稳

结论

该公司正从周期敏感型企业演变为结构脆弱型企业。2024 年高波动的特殊性在于: 在宏观环境平稳背景下, 公司仍出现剧烈利润震荡, 这比宏观下行期的波动更为危险。

核心问题不再是“如何应对周期”，而是“如何修复基本经营稳定性”。公司需要立即停止盲目扩张，转向效率提升，同时重建成本控制能力，寻找盈利稳定性锚点，否则，任何轻微的市场波动都可能导致公司陷入亏损。

五、文本分析

5.1 将年报转化为 txt

```
# 识别并读取年报 PDF 文件列表
report_dir = "annual_reports"

pdf_files = sorted([
    os.path.join(report_dir, f)
    for f in os.listdir(report_dir)
    if f.lower().endswith(".pdf")
])

print("识别到的年报文件: ")
for f in pdf_files:
    print(f)

最后执行于2025-12-14,16: 33: 29, 耗时5毫秒
```

识别到的年报文件:
annual_reports\海南天然橡胶产业集团股份有限公司2022年年度报告.pdf
annual_reports\海南天然橡胶产业集团股份有限公司2023年年度报告.pdf
annual_reports\海南天然橡胶产业集团股份有限公司2024年年度报告.pdf

```
import os
import pdfplumber # 需要先安装: pip install pdfplumber

# 用 pdfplumber 打开 PDF (只做“能不能读”的检查)
print("正在检查PDF文件可读性...")
print("-" * 60)

for pdf_path in pdf_files:
    try:
        with pdfplumber.open(pdf_path) as pdf:
            page_count = len(pdf.pages)
            # 检查第一页是否有文字内容
            first_page = pdf.pages[0]
            text = first_page.extract_text()
            text_length = len(text.strip()) if text else 0

            status = "✓ 可读" if text_length > 0 else "⚠ 无文字内容"

            print(f"{os.path.basename(pdf_path)} | 页数: {page_count:3d} | 状态: {status}")

            if text_length > 0:
                # 显示前几行文字预览
                preview_lines = text.strip().split('\n')[:3]
                print(f"    预览: {' '.join([line.strip() for line in preview_lines])}")

    except Exception as e:
        print(f"{os.path.basename(pdf_path)} | 错误: {type(e).__name__}: {str(e)}")

print("-" * 60)
```

```
# 如果没有PDF文件, 给出提示
if len(pdf_files) == 0:
    print("没有找到PDF文件可供检查")
    print("请确保:")
    print("1. 已创建 'annual_reports' 目录")
    print("2. 目录中包含PDF格式的年报文件")
    print("3. 文件名以.pdf结尾")

# 可选: 提供安装帮助
try:
    import pdfplumber
    print("\npdfplumber 已成功导入")
except ImportError:
    print("\n" + "="*60)
    print("需要安装 pdfplumber 库")
    print("请在命令行中运行: pip install pdfplumber")
    print("或在Jupyter中运行: !pip install pdfplumber")
    print("="*60)
```

最后一次执行是在2025-12-14, 时间16: 34: 59, 时间1.46秒

正在检查PDF文件可读性...

```
-----
海南天然橡胶产业集团股份有限公司2022年年度报告.pdf | 页数: 244 | 状态: ✓ 可读
    预览: 2022年年度报告 公司代码: 601118 公司简称: 海南橡胶 海南天然橡胶产业集团股份有限公司
-----
海南天然橡胶产业集团股份有限公司2023年年度报告.pdf | 页数: 278 | 状态: ✓ 可读
    预览: 2023年年度报告 公司代码: 601118 公司简称: 海南橡胶 海南天然橡胶产业集团股份有限公司
-----
海南天然橡胶产业集团股份有限公司2024年年度报告.pdf | 页数: 248 | 状态: ✓ 可读
    预览: 海南天然橡胶产业集团股份有限公司2024年年度报告 公司代码: 601118 公司简称: 海南橡胶 海南天然橡胶产业集团股份有限公司
-----
```

pdfplumber 已成功导入

```
import os
import re # 新增导入
import pdfplumber

# 逐页读文本 (只读, 不清洗、不保存)
if not pdf_files:
    print("没有找到PDF文件")
else:
    # 示例: 读取第一份年报全文
    pdf_path = pdf_files[0]
    print(f"正在读取: {os.path.basename(pdf_path)}")
    print("-" * 60)

    texts = []

    try:
        with pdfplumber.open(pdf_path) as pdf:
            total_pages = len(pdf.pages)
            print(f"总页数: {total_pages}")

            for i, page in enumerate(pdf.pages, 1):
                # 显示进度 (每10页或最后一页)
                if i % 10 == 0 or i == total_pages:
                    print(f"正在处理第 {i}/{total_pages} 页...")

                text = page.extract_text()
                if text:
                    # 去除多余空白字符
                    text = re.sub(r"\s+", " ", text.strip())
                    texts.append(f"=== 第 {i} 页 ===")
                    texts.append(text)
```

```

full_text = "\n".join(texts)

print("-" * 60)
print(f"字符数: {len(full_text):,}")
print(f"页数: {total_pages}")
print(f"平均每页字符数: {len(full_text)/total_pages:.0f}")

print("\n前500字符预览:")
print("-" * 60)
preview = full_text[:500]
# 确保预览完整结束在一个句子或单词后
if len(full_text) > 500:
    # 找到最后一个空格或标点, 避免截断单词
    last_space = max(preview.rfind(' '), preview.rfind('.'), preview.rfind(','))
    if last_space > 400: # 确保有足够的內容
        preview = preview[:last_space] + " [...]"
print(preview)
print("-" * 60)

# 显示一些统计信息
print("\n文本统计:")
print(f"非空页数: {len([t for t in texts if len(t.strip()) > 10])}")
print(f"总段落数: {len(full_text.split('\n'))}")

# 查找可能的章节标题 (包含"章"、"节"、"目录"等)
chapter_keywords = ["章", "节", "目录", "CHAPTER", "SECTION"]
found_chapters = []
for line in full_text.split('\n'):
    if any(keyword in line for keyword in chapter_keywords):
        if len(line.strip()) < 100: # 标题通常较短
            found_chapters.append(line.strip())

```

```

if found_chapters:
    print("\n检测到的可能章节标题:")
    for i, chapter in enumerate(set(found_chapters[:10]), 1):
        print(f" {i}. {chapter}")

except Exception as e:
    print(f"读取PDF时出错: {type(e).__name__}: {str(e)}")

```

最后执行于 2025年12月14日, 时间16: 36: 53, 37.34秒

正在读取: 海南天然橡胶产业集团股份有限公司2022年年度报告.pdf

总页数: 244

正在处理第 10/244 页...
正在处理第 20/244 页...
正在处理第 30/244 页...
正在处理第 40/244 页...
正在处理第 50/244 页...
正在处理第 60/244 页...
正在处理第 70/244 页...
正在处理第 80/244 页...
正在处理第 90/244 页...
正在处理第 100/244 页...
正在处理第 110/244 页...
正在处理第 120/244 页...
正在处理第 130/244 页...
正在处理第 140/244 页...
正在处理第 150/244 页...
正在处理第 160/244 页...
正在处理第 170/244 页...
正在处理第 180/244 页...
正在处理第 190/244 页...
正在处理第 200/244 页...
正在处理第 210/244 页...
正在处理第 220/244 页...
正在处理第 230/244 页...
正在处理第 240/244 页...
正在处理第 244/244 页...

字符数: 235,137

页数: 244

平均每页字符数: 964

前500字符预览：

=== 第 1 页 ===
2022年年度报告 公司代码：601118 公司简称：海南橡胶 海南天然橡胶产业集团股份有限公司 2022 年年度报告 1 / 244
=== 第 2 页 ===
2022年年度报告 重要提示 一、 本公司董事会、监事会及董事、监事、高级管理人员保证年度报告内容的真实性、准确性、 完整性，不存在虚假记载、误导性陈述或重大遗漏，并承担个别和连带的法律责任。 二、 公司全体董事出席董事会会议。 三、 中审众环会计师事务所（特殊普通合伙）为本公司出具了保留意见审计报告，本公司董事会、 监事会对相关事项已有详细说明，请投资者注意阅读。 四、 公司负责人艾轶伦、主管会计工作负责人孙卫良及会计机构负责人（会计主管人员）孙和亮 声明：保证年度报告中财务报告的真实、准确、完整。 五、 董事会决议通过的本报告期利润分配预案或公积金转增股本预案 公司拟定2022年度利润分配预案为：不派发现金红利、不送红股、不以公积金转增股本。 六、 前瞻性陈述的风险声明 ☒适用 ☐不适用 本报告中所涉及的未来计划、发展战略等前瞻性描述不构成公司对投资者的承诺，敬请投资 者注意投资风险。 七、 [...]

文本统计：
非空页数：488
总段落数：1

检测到的可能章节标题：
1. 2022年年度报告 八、 股份回购在报告期的具体实施情况 ☐适用 ☒不适用 第八节 优先股相关情况 ☐适用 ☒不适用 72 / 244

5.2 进行年报文本的清洗

同时读入五份年报

```
all_reports_text = {}

for pdf_path in pdf_files:
    year = os.path.basename(pdf_path).replace(".pdf", "")
    texts = []

    with pdfplumber.open(pdf_path) as pdf:
        for page in pdf.pages:
            text = page.extract_text()
            if text:
                text = re.sub(r"\s+", " ", text)
                texts.append(text)

    full_text = "\n".join(texts)
    all_reports_text[year] = full_text

print(year, "字符数: ", len(full_text))
```

最后一次执行是在2025-12-14, 成绩16: 39: 14, 成绩1分48.86秒

海南天然橡胶产业集团股份有限公司2022年年度报告 字符数: 231341
海南天然橡胶产业集团股份有限公司2023年年度报告 字符数: 254595
海南天然橡胶产业集团股份有限公司2024年年度报告 字符数: 250372

PDF → TXT（逐年保存 + 统计报告）

1) 输出目录（没有就创建）

```
out_dir = "annual_reports/data/txt"
os.makedirs(out_dir, exist_ok=True)
```

stats = [] # 记录每年的统计信息

```
for pdf_path in pdf_files:
    file_name = os.path.basename(pdf_path)

    # 2) 从文件名里提取年份（简单方法：找连续4位数字）
    m = re.search(r"(20\d{2})", file_name)
    year = m.group(1) if m else file_name.replace(".pdf", "")

    texts = []
    total_pages = 0
    empty_pages = 0

    with pdfplumber.open(pdf_path) as pdf:
        total_pages = len(pdf.pages)

        for page in pdf.pages:
            text = page.extract_text()
            if text and text.strip():
                # 简单清洗：合并空白
                text = re.sub(r"\s+", " ", text)
                texts.append(text)
            else:
                empty_pages += 1

    full_text = "\n".join(texts)
```

```

# 3) 保存为 txt
txt_path = os.path.join(out_dir, f"{year}.txt")
with open(txt_path, "w", encoding="utf-8") as f:
    f.write(full_text)

# 4) 统计信息
word_count = len(full_text)
success_rate = (total_pages - empty_pages) / total_pages if total_pages > 0 else 0
empty_ratio = empty_pages / total_pages if total_pages > 0 else 0

stats.append([year, total_pages, empty_pages, round(empty_ratio, 4), round(success_rate, 4), word_count])

print(f"✅ {year} 年报已保存: {txt_path} | 字符数={word_count} | 空页比例=(empty_ratio:.2%)")

# 5) 输出汇总报告
print("\n==== PDF→TXT 汇总报告 =====")
print("year | 总页数 | 空页数 | 空页比例 | 提取成功率 | 字符数")
for row in stats:
    print(row)

```

最后执行于2025-12-14, 16: 41: 22, 成绩1分49.24秒

```

✅ 2022 年报已保存: annual_reports/data/txt/2022.txt | 字符数=231341 | 空页比例=0.00%
✅ 2023 年报已保存: annual_reports/data/txt/2023.txt | 字符数=254595 | 空页比例=0.00%
✅ 2024 年报已保存: annual_reports/data/txt/2024.txt | 字符数=250372 | 空页比例=0.00%

```

```

==== PDF→TXT 汇总报告 =====
year | 总页数 | 空页数 | 空页比例 | 提取成功率 | 字符数
['2022', 244, 0, 0.0, 1.0, 231341]
['2023', 278, 0, 0.0, 1.0, 254595]
['2024', 248, 0, 0.0, 1.0, 250372]

```

```

# 逐年分词 + 输出 Top30 (按年 + 5年合并)
txt_dir = "annual_reports/data/txt"
stop_path = "stopwords.txt"

# 1) 读取停用词
with open(stop_path, "r", encoding="utf-8") as f:
    stopwords = set([line.strip() for line in f if line.strip()])

# 2) 找到所有txt
txt_files = sorted([os.path.join(txt_dir, f) for f in os.listdir(txt_dir) if f.endswith(".txt")])
print("识别到的TXT: ", [os.path.basename(x) for x in txt_files])

def tokenize(text):
    """中文分词 + 基础清洗 (零基础友好版)"""
    text = re.sub(r"[a-zA-Z0-9]+", " ", text) # 去英数
    text = re.sub(r"^[u4e00-\u9fa5]+", " ", text) # 只保留中文
    words = jieba.lcut(text)
    # 过滤: 停用词、长度<2
    words = [w for w in words if (len(w) >= 2) and (w not in stopwords)]
    return words

year_top = {} # 每年Top30
all_words = [] # 合并词

for path in txt_files:
    year = os.path.basename(path).replace(".txt", "")
    with open(path, "r", encoding="utf-8") as f:
        text = f.read()

    words = tokenize(text)
    all_words.extend(words)

```

```

top30 = Counter(words).most_common(30)
year_top[year] = top30

print(f"\n==== {year} Top30 高频词 =====")
for w, c in top30:
    print(w, c)

# 合并Top30
print("\n==== 五年合并 Top30 高频词 =====")
top30_all = Counter(all_words).most_common(30)
for w, c in top30_all:
    print(w, c)

```

最后执行于2025-12-14, 时间16: 46: 48, 时间1.85秒

识别到的TXT: ['2022.txt', '2023.txt', '2024.txt']

===== 2022 Top30 高频词 =====

海南 625
投资 497
资产 456
橡胶 399
农垦 318
价值 312
企业 259
项目 247
负债 238
收益 221
交易 218
股东 217
合并 201
变动 197
确认 195
公允 194
股权 194
损益 183
天然橡胶 173
经营 167
计入 154
现金 153
海南省 149
成本 149
计量 147
当期 145
控制 144
控股 143
账面 143

处置 139

===== 2023 Top30 高频词 =====

海南 657
资产 445
投资 421
橡胶 415
农垦 373
项目 318
价值 303
股东 254
负债 239
确认 224
变动 211
企业 208
收益 198
控股 195
经营 189
公允 187
股权 179
损失 175
天然橡胶 170
交易 169
海南省 167
损益 166
现金 161
账面 161
计量 160
收入 159
下属 152
管理 150
成本 149
合并 147

===== 2024 Top30 高频词 =====
海南 798
天然橡胶 460
资产 415
投资 383
产业 366
橡胶 336
项目 322
农垦 321
价值 294
集团股份 276
海垦 249
控股集团 243
负债 230
确认 225
变动 214
企业 199
收益 194
经营 180
计提 175
损失 174
公允 170
账面 164
交易 160
损益 159
收入 157
股东 155
现金 154
计量 152
下属 152
坏账 152

===== 五年合并 Top30 高频词 =====
海南 2080
资产 1316
投资 1301
橡胶 1150
农垦 1012
价值 909
项目 887
天然橡胶 803
负债 707
企业 666
确认 644
股东 626
变动 622
收益 613
产业 574
公允 551
交易 547
经营 536
损益 508
股权 502
控股集团 496
损失 487
海垦 481
合并 468
账面 468
现金 468
海南省 464
计量 459
收入 448
成本 443

5.3 进行词云图分析

```

# 图像保存目录
fig_dir = "annual_reports/data/fig"
os.makedirs(fig_dir, exist_ok=True)

# 读取停用词
with open("annual_reports/data/stopwords.txt", "r", encoding="utf-8") as f:
    stopwords = set([line.strip() for line in f if line.strip()])

def tokenize(text):
    text = re.sub(r"[a-zA-Z0-9]+", " ", text)
    text = re.sub(r"^\u4e00-\u9fa5+", " ", text)
    words = jieba.lcut(text)
    return [w for w in words if len(w) >= 2 and w not in stopwords]

def make_wordcloud(words, title, save_path):
    freqs = dict(Counter(words))

    wc = WordCloud(
        font_path="C:/Windows/Fonts/simhei.ttf", # ★直接写死
        width=900,
        height=500,
        background_color="white"
    ).generate_from_frequencies(freqs)

    plt.figure(figsize=(12, 6))
    plt.imshow(wc, interpolation="bilinear")
    plt.axis("off")
    plt.title(title)
    plt.tight_layout()
    plt.savefig(save_path, dpi=200)
    plt.show()

# 逐年词云
txt_dir = "annual_reports/data/txt"
txt_files = sorted([f for f in os.listdir(txt_dir) if f.endswith(".txt")])

```

```

all_words = []

for fname in txt_files:
    year = fname.replace(".txt", "")
    with open(os.path.join(txt_dir, fname), "r", encoding="utf-8") as f:
        text = f.read()

    words = tokenize(text)
    all_words.extend(words)

    save_path = os.path.join(fig_dir, f"wordcloud_{year}.png")
    make_wordcloud(words, f"{year} 年报词云图", save_path)

# 五年合并词云
make_wordcloud(
    all_words,
    "五年合并年报词云图",
    os.path.join(fig_dir, "wordcloud_all_5y.png")
)

```

最后执行时间为2026-01-04,22: 29: 44, 耗时122毫

```

-----
FileNotFoundError                                Traceback (most recent call last)
Cell In[9], line 6
      3 os.makedirs(fig_dir, exist_ok=True)
      5 # 读取停用词
----> 6 with open("annual_reports/data/stopwords.txt", "r", encoding="utf-8") as f:
      7     stopwords = set([line.strip() for line in f if line.strip()])
      9 def tokenize(text):

File C:\miniconda3\Lib\site-packages\IPython\core\interactiveshell.py:343, in _modified_open(file, *args, **kwargs)
    336 if file in {0, 1, 2}:
    337     raise ValueError(
    338         f"IPython won't let you open fd={file} by default "
    339         "as it is likely to crash IPython. If you know what you are doing, "
    340         "you can use builtins' open."
    341     )
--> 343 return io_open(file, *args, **kwargs)

FileNotFoundError: [Errno 2] No such file or directory: 'annual_reports/data/stopwords.txt'

```

```

import os
import re
from collections import Counter
import jieba
from wordcloud import WordCloud
import matplotlib.pyplot as plt

# 创建目录结构
data_dir = "annual_reports/data"
txt_dir = "annual_reports/data/txt"
fig_dir = "annual_reports/data/fig"

os.makedirs(data_dir, exist_ok=True)
os.makedirs(txt_dir, exist_ok=True)
os.makedirs(fig_dir, exist_ok=True)

# 创建或读取停用词文件
stopwords_path = "annual_reports/data/stopwords.txt"
if not os.path.exists(stopwords_path):
    print(f"警告: {stopwords_path} 不存在, 使用空停用词列表")
    stopwords = set()
else:
    with open(stopwords_path, "r", encoding="utf-8") as f:
        stopwords = set([line.strip() for line in f if line.strip()])
    print(f"已加载 {len(stopwords)} 个停用词")

def tokenize(text):
    # 移除英文和数字
    text = re.sub(r"[a-zA-Z0-9]+", " ", text)
    # 只保留中文
    text = re.sub(r"^\u4e00-\u9fa5+", " ", text)
    # 使用jieba分词

```

```

    words = jieba.lcut(text)
    # 过滤: 长度>=2且不在停用词中
    return [w for w in words if len(w) >= 2 and w not in stopwords]

def make_wordcloud(words, title, save_path):
    if not words:
        print(f"警告: {title} 没有词语可生成词云")
        return

    # 统计词频
    freqs = dict(Counter(words))

    # 设置字体路径 (如果Windows路径不存在, 尝试其他方式)
    font_path = "C:/Windows/Fonts/simhei.ttf"
    if not os.path.exists(font_path):
        # 尝试其他可能的字体路径
        font_path = None # WordCloud使用默认字体
        print("使用默认字体")

    # 生成词云
    wc = WordCloud(
        font_path=font_path,
        width=900,
        height=500,
        background_color="white"
    ).generate_from_frequencies(freqs)

    # 显示和保存
    plt.figure(figsize=(12, 6))
    plt.imshow(wc, interpolation="bilinear")
    plt.axis("off")
    plt.title(title)
    plt.tight_layout()

```

最后执行时间为2026-01-04, 时间22: 30: 19, 时间为22.03秒





5.4 从年报提取 MD&A 文本

提取MD&A计算情感

提取思路：

TXT 优先（沿用你原有管线）

但 TXT 起点不再用 last match，而是用**“候选起点打分”**（避免 2021/2023 这种“请参阅第四节...” 引用句误命中）

如果 TXT 提取出来太短（默认 < 20000 字），自动判定为失败并回退到 PDF 提取

PDF 提取同样用打分避免目录/引用句

输出统一到：annual_reports/data/mdna_full_3/{year}_mdna_full_3.txt

```
pip install PyMuPDF
```

最后执行时间：2026-01-04 22: 48: 12. 时间50.66秒

^C

Note: you may need to restart the kernel to use updated packages.

```
import re
from pathlib import Path
import os

# =====
# 1) 路径配置（按你项目）
# =====
TXT_DIR = Path("annual_reports/data/txt")          # 你已确认
OUT_DIR = Path("annual_reports/data/mdna_full_text") # 输出目录（可改）

YEARS = [2020, 2021, 2022, 2023, 2024]

# TXT 提取出来如果少于这个字符数，认为“误命中/截断”
MIN_VALID_LEN = 20000

# =====
# 2) 通用清洗
# =====
def normalize_keep_lines(text: str) -> str:
    """清洗文本但保留换行"""
    text = text.replace("\u3000", " ")
    text = re.sub(r"[ \t]+", " ", text)
    text = re.sub(r"\n{2,}", "\n", text)
    return text.strip()

# =====
# 3) TXT：候选起点打分，避免引用句
# =====
def best_start_by_signals_in_txt(full_text: str, start_pat: str) -> int:
    cands = [m.start() for m in re.finditer(start_pat, full_text)]
    if not cands:
        return -1

    signals = [
        "经营情况讨论与分析", "报告期内", "主营业务", "收入", "同比", "毛利率",
        "晶圆代工", "产能利用率", "风险因素", "核心竞争力", "研发投入"
    ]

    best_i, best_score = cands[0], -10
    for i in cands:
        window = full_text[i:i + 2500]
        score = sum(1 for s in signals if s in window)

        # 引用句惩罚
        near = full_text[max(0, i - 80): i + 80]
        if "请参阅" in near or "详见" in near:
            score -= 2

        # 如果很快就进入“公司治理”，基本就是错的
        if "第六节 公司治理" in window[:900] or "公司治理" in window[:900]:
            score -= 4

        if score > best_score:
            best_score = score
            best_i = i

    return best_i
```

```

def extract_mdna_from_txt(full_text: str, year: int) -> str:
    """从纯文本提取管理层讨论与分析部分"""
    full_text = normalize_keep_lines(full_text)

    # 2020: 第五节 经营情况讨论与分析
    start_pat_2020 = r"第\s*五\s*节\s*经\s*营\s*情\s*况\s*讨\s*论\s*与\s*分\s*析"
    # 2021+: 第四节 管理层讨论 与/ 及/ 和 分析
    start_pat_4 = r"第\s*四\s*节\s*管\s*理\s*层\s*讨\s*论\s*(与|及|和)\s*分\s*析"
    # 兜底（正文常见）
    fallback = r"报\s*告\s*期\s*内\s*主\s*要\s*经\s*营\s*情\s*况"

    if year == 2020:
        start_idx = best_start_by_signals_in_txt(full_text, start_pat_2020)
    else:
        start_idx = best_start_by_signals_in_txt(full_text, start_pat_4)

    if start_idx == -1:
        m = re.search(fallback, full_text)
        start_idx = m.start() if m else -1

    if start_idx == -1:
        return ""

    mdna = full_text[start_idx:start_idx + 300000]

    # 终点: 优先第五节董事会报告, 再退到第六节
    end_pats = [
        r"第\s*五\s*节\s*董\s*事\s*会\s*报\s*告",
        r"第\s*五\s*节\s*董\s*事\s*会\s*报\s*告",
        r"第\s*六\s*节",
        r"第\s*六\s*节",
    ]

```

```

cut = None
for ep in end_pats:
    m = re.search(ep, mdna[200:])
    if m:
        cut = 200 + m.start()
        break

if cut:
    mdna = mdna[:cut]

return mdna.strip()

# =====
# 4) 主流程: 批量提取 (仅TXT版)
# =====
OUT_DIR.mkdir(parents=True, exist_ok=True)

print("开始提取管理层讨论与分析 (MD&A) ...")
print("=" * 60)

for year in YEARS:
    txt_path = TXT_DIR / f"{year}.txt"
    out_path = OUT_DIR / f"{year}_mdna.txt"

```

```
# 检查TXT文件是否存在
if txt_path.exists():
    try:
        text = txt_path.read_text(encoding="utf-8")
        mdna = extract_mdna_from_txt(text, year)
        used = "TXT"
    except Exception as e:
        print(f"读取 {txt_path} 时出错: {e}")
        mdna = ""
    else:
        print(f"🚩 {txt_path} 不存在, 跳过")

# 保存
if mdna:
    out_path.write_text(mdna, encoding="utf-8")

# 报告
status = "✅" if len(mdna) >= MIN_VALID_LEN else "🚩"
print(f"{status} {year} -> {used} | 字符数: {len(mdna):,}")

if len(mdna) < MIN_VALID_LEN:
    print(f"⚠️ 警告: 提取内容可能不完整 (少于{MIN_VALID_LEN:,}字符)")

# 显示前几句话预览
preview = mdna[:300].replace("\n", " ")
if len(mdna) > 300:
    preview += "... "
print(f"👁️ 预览: {preview}")
else:
    print(f"❌ {year} -> 提取失败")

print("-" * 60)

print("-" * 60)

print("\n" + "=" * 60)
print(f"提取完成! 结果保存在: {OUT_DIR}")

2026-01-04 22: 48: 13. 成绩1.24秒失败
开始提取管理层讨论与分析 (MD&A) ...
=====
🚩 annual_reports\data\txt\2020.txt 不存在, 跳过
❌ 2020 -> 提取失败
-----
🚩 annual_reports\data\txt\2021.txt 不存在, 跳过
❌ 2021 -> 提取失败
-----
✅ 2022 -> TXT | 字符数: 49,228
预览: 报告期内主要经营情况 报告期内, 公司实现营业收入153.71亿元, 自产橡胶产品11.62万吨, 归属于上市公司股 东净利润0.76亿元。报告期末, 资产总额为22
3.29亿元, 归属于上市公司股东净资产为93.76 亿元。(一) 主营业务分析 1. 利润表及现金流量表相关科目变动分析表 单位: 元 币种: 人民币 科目 变动比例 本期
数 上年同期数 (%) 营业收入 15,371,271,093.62 15,207,422,670.09 1.08 营业成本 15,038,018,950.67 14,576,418,543.60 3.17 销售费用 191,363,31
3.37 215,608,349....
-----
✅ 2023 -> TXT | 字符数: 60,616
预览: 报告期内主要经营情况 报告期内, 公司实现营业收入376.87亿元, 自产橡胶产品13.95万吨, 归属于上市公司股东 净利润2.97亿元。报告期末, 资产总额为33
9.50亿元, 归属于上市公司股东净资产为98.25亿 元。 13 / 290 2023年年度报告 (一) 主营业务分析 1. 利润表及现金流量表相关科目变动分析表 单位: 元 币种:
人民币 科目 本期数 上年同期数 变动比例 (%) 营业收入 37,687,250,562.10 15,371,271,093.62 145.18 营业成本 36,887,078,835.89 15,038,018,950.67
145.29 销售费用 232,0...
-----
✅ 2024 -> TXT | 字符数: 54,903
预览: 报告期内主要经营情况 报告期内, 公司实现营业收入496.73亿元, 自产橡胶产品12.13万吨, 归属于上市公司股 东净利润1.03亿元。报告期末, 资产总额为36
0.70亿元, 归属于上市公司股东净资产98.46亿 元。(一) 主营业务分析 1、利润表及现金流量表相关科目变动分析表 单位: 元 币种: 人民币 变动比例 科目 本期数
上年同期数 (%) 营业收入 49,672,606,732.67 37,687,250,562.10 31.80 营业成本 48,024,221,191.93 36,887,078,835.89 30.19 销售费用 314,584,246.
34 232,013,552....
-----

提取完成! 结果保存在: annual_reports\data\mdna_full_text
```

5.5 进行情感分析


```

import re
from pathlib import Path
import os
import jieba
import pandas as pd

# =====
# 1. 配置路径
# =====
TXT_DIR = Path("annual_reports/data/txt") # 原始文本目录
MDNA_DIR = Path("annual_reports/data/mdna_full_text") # MD&A输出目录
SENTIMENT_DIR = Path("annual_reports/data/dict") # 情感词典目录

# 确保目录存在
MDNA_DIR.mkdir(parents=True, exist_ok=True)
SENTIMENT_DIR.mkdir(parents=True, exist_ok=True)

print("📁 目录配置:")
print(f"  原始文本目录: {TXT_DIR}")
print(f"  MD&A输出目录: {MDNA_DIR}")
print(f"  情感词典目录: {SENTIMENT_DIR}")

# =====
# 2. 创建或加载情感词典
# =====
def create_default_sentiment_dict():
    """创建默认的情感词典"""
    # 正向词典
    positive_words = [
        "积极", "乐观", "增长", "提升", "增加", "上升", "提高", "改善", "好转", "成功",
        "良好", "优秀", "优异", "出色", "突出", "显著", "明显", "大幅", "大幅增长", "进步",
        "发展", "拓展", "扩张", "扩大", "增强", "加强", "巩固", "稳定", "稳固", "创新",
        "创造", "领先", "引领", "突破", "超越", "卓越", "完美", "完善", "优化", "高效",
        "高质", "高质量", "高水平", "高标准", "高效益", "高回报", "高价值", "受益", "获利", "盈利",
        "盈余", "利润", "收益", "收入", "营收", "业绩", "绩效", "增长点", "增长性", "增长空间",
        "增长潜力", "增长动力", "机会", "机遇", "前景", "未来", "希望", "光明", "灿烂", "辉煌",
        "繁荣", "兴旺", "满意", "喜悦", "高兴", "欣喜", "欣慰", "鼓舞", "振奋", "激动", "感动",
        "感激", "合作", "协同", "共赢", "共享", "互助", "支持", "帮助", "扶持", "促进", "推动"
    ]

    # 负向词典
    negative_words = [
        "消极", "悲观", "下降", "降低", "减少", "下滑", "下跌", "恶化", "变差", "失败",
        "不良", "差劣", "糟糕", "落后", "滞后", "不足", "欠缺", "缺陷", "问题", "困难",
        "挑战", "压力", "风险", "危险", "危机", "威胁", "隐患", "不利", "负面影响", "亏损",
        "损失", "亏本", "赤字", "亏损额", "亏损面", "亏损点", "亏损状态", "疲软", "疲弱", "疲态",
        "低迷", "萧条", "萎缩", "收缩", "紧缩", "压缩", "削减", "冲突", "矛盾", "纠纷",
        "争议", "分歧", "对立", "对抗", "抵制", "担忧", "忧虑", "焦虑", "担心", "不安",
        "恐慌", "恐惧", "畏惧", "害怕", "失望", "失落", "沮丧", "灰心", "泄气", "气馁",
        "无助", "无奈", "无力", "无能", "混乱", "杂乱", "无序", "紊乱", "失序", "失控",
        "失范", "失准", "失衡", "失调", "紧张", "激烈", "激烈竞争", "激烈对抗", "激烈冲突", "激烈争夺"
    ]

    # 保存到文件
    positive_path = SENTIMENT_DIR / "positive.txt"
    negative_path = SENTIMENT_DIR / "negative.txt"

    with open(positive_path, "w", encoding="utf-8") as f:
        f.write("\n".join(positive_words))

    with open(negative_path, "w", encoding="utf-8") as f:
        f.write("\n".join(negative_words))

    print(f"✅ 已创建情感词典:")

```

```

print(f"    正向词典: {positive_path} ({len(positive_words)} 个词)")
print(f"    负向词典: {negative_path} ({len(negative_words)} 个词)")

return set(positive_words), set(negative_words)

def load_sentiment_dict():
    """加载情感词典"""
    positive_path = SENTIMENT_DIR / "positive.txt"
    negative_path = SENTIMENT_DIR / "negative.txt"

    positive_words = set()
    negative_words = set()

    # 加载正向词典
    if positive_path.exists():
        try:
            with open(positive_path, "r", encoding="utf-8") as f:
                positive_words = set(line.strip() for line in f if line.strip())
            print(f"✅ 加载正向词典: {len(positive_words)} 个词")
        except:
            with open(positive_path, "r", encoding="gbk") as f:
                positive_words = set(line.strip() for line in f if line.strip())
            print(f"✅ 从GBK加载正向词典: {len(positive_words)} 个词")
    else:
        print("❌ 正向词典不存在")

    # 加载负向词典
    if negative_path.exists():
        try:
            with open(negative_path, "r", encoding="utf-8") as f:
                negative_words = set(line.strip() for line in f if line.strip())
            print(f"✅ 加载负向词典: {len(negative_words)} 个词")
        except:
            with open(negative_path, "r", encoding="gbk") as f:
                negative_words = set(line.strip() for line in f if line.strip())
            print(f"✅ 从GBK加载负向词典: {len(negative_words)} 个词")
    else:
        print("❌ 负向词典不存在")

    return positive_words, negative_words

# 加载或创建情感词典
if (SENTIMENT_DIR / "positive.txt").exists() and (SENTIMENT_DIR / "negative.txt").exists():
    positive_words, negative_words = load_sentiment_dict()
else:
    print("⚠️ 情感词典不存在, 创建默认词典...")
    positive_words, negative_words = create_default_sentiment_dict()

# =====
# 3. 从原始文本提取MD&A的函数
# =====
def normalize_text(text):
    """清洗文本"""
    text = re.sub(r'\s+', ' ', text) # 合并空白字符
    text = re.sub(r'^\u4e00-\u9fa5,。；：！？、（）《》【】', ' ', text) # 保留中文和标点
    return text.strip()

def extract_mdna_from_text(text, year):
    """从年报文本中提取管理层讨论与分析部分"""
    # 根据不同年份使用不同的关键词
    if year == 2022 or year == 2023 or year == 2024:
        # 近年常见格式
        start_patterns = [
            r"第\s*四\s*节\s*管理层讨论与分析",
            r"第四节\s*管理层讨论与分析",
            r"管理层讨论与分析",

```

```

        r"经营情况讨论与分析"
    ]
else:
    # 更早年份
    start_patterns = [
        r"第\s*五\s*节\s*经营情况讨论与分析",
        r"第五节\s*经营情况讨论与分析"
    ]

# 寻找开始位置
start_idx = -1
for pattern in start_patterns:
    match = re.search(pattern, text)
    if match:
        start_idx = match.start()
        print(f"    使用模式 '{pattern}' 找到开始位置")
        break

if start_idx == -1:
    print(f"    警告: 未找到MD&A开始标记")
    return ""

# 截取从开始位置到文本结束
mdna_text = text[start_idx:]

# 寻找结束位置 (通常是下一节开始)
end_patterns = [
    r"第\s*五\s*节\s*董事会报告",
    r"第五节\s*董事会报告",
    r"第\s*六\s*节",
    r"第六节",
    r"重要事项",
    r"公司治理"

```

```

]

end_idx = -1
for pattern in end_patterns:
    match = re.search(pattern, mdna_text[1000:]) # 跳过头1000字符避免匹配到标题
    if match:
        end_idx = 1000 + match.start()
        print(f"    使用模式 '{pattern}' 找到结束位置")
        break

if end_idx != -1:
    mdna_text = mdna_text[:end_idx]

return normalize_text(mdna_text)

def extract_all_mdna():
    """提取所有年份的MD&A"""
    print("\n🔍 开始提取MD&A内容...")

    # 查找所有txt文件
    txt_files = sorted(TXT_DIR.glob("*.txt"))
    print(f"找到 {len(txt_files)} 个文本文件:")

    mdna_texts = {}

    for txt_file in txt_files:
        # 从文件名提取年份
        match = re.search(r'(\d{4})', txt_file.stem)
        if not match:
            print(f"❌ 无法从文件名提取年份: {txt_file.name}")
            continue

```



```

year = int(match.group(1))
print(f"\n 📅 处理 {year} 年年报...")

try:
    # 读取文本
    with open(txt_file, 'r', encoding='utf-8') as f:
        text = f.read()

    print(f" 原始文本长度: {len(text):,} 字符")

    # 提取MD&A
    mdna_text = extract_mdna_from_text(text, year)

    if mdna_text:
        # 保存MD&A到文件
        output_file = MDNA_DIR / f"{year}_mdna.txt"
        with open(output_file, 'w', encoding='utf-8') as f:
            f.write(mdna_text)

        mdna_texts[year] = mdna_text
        print(f"✅ 提取成功: {len(mdna_text):,} 字符")
        print(f" 已保存到: {output_file}")
    else:
        print(f"❌ 提取失败, 未找到MD&A内容")

except Exception as e:
    print(f"❌ 处理 {txt_file} 时出错: {e}")

return mdna_texts

```

```

# =====
# 4. 情感分析函数
# =====
def sentiment_analysis(text, positive_words, negative_words):
    """进行情感分析"""
    if not text:
        return 0, 0, 0, 0

    # 分词
    words = jieba.lcut(text)
    total_words = len(words)

    # 统计情感词
    pos_count = sum(1 for word in words if word in positive_words)
    neg_count = sum(1 for word in words if word in negative_words)

    # 计算情感得分
    if pos_count + neg_count == 0:
        sentiment_score = 0
    else:
        sentiment_score = (pos_count - neg_count) / (pos_count + neg_count)

    # 计算每万字词频
    pos_density = pos_count / total_words * 10000 if total_words > 0 else 0
    neg_density = neg_count / total_words * 10000 if total_words > 0 else 0

    return sentiment_score, pos_count, neg_count, pos_density, neg_density

def analyze_all_years(mdna_texts):
    """分析所有年份的情感"""
    print("\n 📅 开始情感分析...")

```

```

results = []

for year, text in sorted(mdna_texts.items()):
    print(f"\n📄 分析 {year} 年...")
    print(f"    文本长度: {len(text):,} 字符")

    # 情感分析
    score, pos_count, neg_count, pos_density, neg_density = sentiment_analysis(
        text, positive_words, negative_words
    )

    print(f"    正向词数: {pos_count}")
    print(f"    负向词数: {neg_count}")
    print(f"    正向密度: {pos_density:.1f} 词/万字")
    print(f"    负向密度: {neg_density:.1f} 词/万字")
    print(f"    情感得分: {score:.3f}")

    if score > 0.1:
        sentiment_label = "积极"
    elif score < -0.1:
        sentiment_label = "消极"
    else:
        sentiment_label = "中性"

    print(f"    情感倾向: {sentiment_label}")

    results.append({
        'year': year,
        'sentiment_score': score,
        'positive_count': pos_count,
        'negative_count': neg_count,
        'positive_density': pos_density,
        'negative_density': neg_density,
        'sentiment_label': sentiment_label,
        'text_length': len(text)
    })

return results

# =====
# 5. 主程序
# =====
def main():
    print("=" * 60)
    print("年报MD&A提取与情感分析系统")
    print("=" * 60)

    # 步骤1: 提取MD&A
    print("\n📌 第一步: 提取MD&A内容")
    mdna_texts = extract_all_mdna()

    if not mdna_texts:
        print("\n❌ 未成功提取任何MD&A内容, 程序结束")
        return

    print(f"\n✅ 成功提取 {len(mdna_texts)} 年的MD&A内容")

    # 步骤2: 情感分析
    print("\n📊 第二步: 情感分析")
    results = analyze_all_years(mdna_texts)

    # 创建DataFrame
    df = pd.DataFrame(results)
    df = df.sort_values('year').reset_index(drop=True)

```

```
# 计算趋势
if len(df) > 1:
    first_score = df.iloc[0]['sentiment_score']
    last_score = df.iloc[-1]['sentiment_score']
    trend = "上升" if last_score > first_score else "下降"
    print(f"\n📈 情感得分趋势: {trend} ({first_score:.3f} → {last_score:.3f})")

# 保存结果
output_csv = Path("annual_reports/data") / "sentiment_analysis_results.csv"
df.to_csv(output_csv, index=False, encoding='utf-8-sig')
print(f"\n💾 分析结果已保存到: {output_csv}")

# 显示汇总结果
print("\n" + "=" * 60)
print("📊 情感分析汇总结果")
print("=" * 60)

# 格式化显示
display_df = df.copy()
display_df['sentiment_score'] = display_df['sentiment_score'].apply(lambda x: f"{x:.3f}")
display_df['positive_density'] = display_df['positive_density'].apply(lambda x: f"{x:.1f}")
display_df['negative_density'] = display_df['negative_density'].apply(lambda x: f"{x:.1f}")

print(display_df[['year', 'sentiment_score', 'positive_count', 'negative_count',
                  'positive_density', 'negative_density', 'sentiment_label']].to_string(index=False))

# 建议
print("\n💡 分析建议:")
for _, row in df.iterrows():
    year = row['year']
    score = row['sentiment_score']

    if score > 0.3:
        print(f" {year}年: 情感非常积极, 可关注其积极因素")
    elif score > 0.1:
        print(f" {year}年: 情感较为积极")
    elif score > -0.1:
        print(f" {year}年: 情感中性, 需结合其他指标分析")
    elif score > -0.3:
        print(f" {year}年: 情感较为消极, 需关注风险因素")
    else:
        print(f" {year}年: 情感非常消极, 需谨慎分析")

print("\n✅ 分析完成!")

# 运行主程序
if __name__ == "__main__":
    main()
```

最后执行于2026年1月4日, 23: 07: 41, 时间1.70秒

📁 目录配置:

原始文本目录: annual_reports\data\txt
MD&A输出目录: annual_reports\data\mdna_full_text
情感词典目录: annual_reports\data\dict

✅ 加载正向词典: 10 个词
✅ 加载负向词典: 10 个词

=====

年报MD&A提取与情感分析系统

=====

1 第一步: 提取MD&A内容

🔍 开始提取MD&A内容...

找到 4 个文本文件:

📄 处理 2022 年年报...

原始文本长度: 231,341 字符
使用模式 '管理层讨论与分析' 找到开始位置
使用模式 '第\s*六\s*节' 找到结束位置
✅ 提取成功: 62,042 字符
已保存到: annual_reports\data\mdna_full_text\2022_mdna.txt

📄 处理 2023 年年报...

原始文本长度: 254,595 字符
使用模式 '管理层讨论与分析' 找到开始位置
使用模式 '第\s*六\s*节' 找到结束位置
✅ 提取成功: 72,896 字符
已保存到: annual_reports\data\mdna_full_text\2023_mdna.txt

📄 处理 2024 年年报...

原始文本长度: 250,372 字符
使用模式 '管理层讨论与分析' 找到开始位置
使用模式 '第\s*六\s*节' 找到结束位置
✅ 提取成功: 68,402 字符
已保存到: annual_reports\data\mdna_full_text\2024_mdna.txt

成功提取 3 年的MD&A内容

第二步：情感分析

开始情感分析...

分析 2022 年...
文本长度：62,042 字符
正向词数：0
负向词数：0
正向密度：0.0 词/万字
负向密度：0.0 词/万字
情感得分：0.000
情感倾向：中性

分析 2023 年...
文本长度：72,896 字符
正向词数：0
负向词数：0
正向密度：0.0 词/万字
负向密度：0.0 词/万字
情感得分：0.000
情感倾向：中性

分析 2024 年...
文本长度：68,402 字符
正向词数：0
负向词数：0
正向密度：0.0 词/万字
负向密度：0.0 词/万字
情感得分：0.000
情感倾向：中性

情感得分趋势：下降 (0.000 → 0.000)

分析结果已保存到：annual_reports\data\sentiment_analysis_results.csv

情感分析汇总结果

year	sentiment_score	positive_count	negative_count	positive_density	negative_density	sentiment_label
2022	0.000	0	0	0.0	0.0	中性
2023	0.000	0	0	0.0	0.0	中性
2024	0.000	0	0	0.0	0.0	中性

分析建议：
2022年：情感中性，需结合其他指标分析
2023年：情感中性，需结合其他指标分析
2024年：情感中性，需结合其他指标分析

分析完成!

5.6 画出情感的折线图

```
import os

# 准备情感词典
# 情感词典所在目录
sentiment_dir = "annual_reports/data/dict" # 你的实际路径

print(f"情感词典目录: {sentiment_dir}")
print(f"目录是否存在: {os.path.exists(sentiment_dir)}")

if os.path.exists(sentiment_dir):
    print("目录内容:", os.listdir(sentiment_dir))

# 尝试用UTF-8编码读取
positive_path = os.path.join(sentiment_dir, "positive.txt")
negative_path = os.path.join(sentiment_dir, "negative.txt")

print(f"\n正向词典文件: {positive_path}")
print(f"文件是否存在: {os.path.exists(positive_path)}")

print(f"\n负向词典文件: {negative_path}")
print(f"文件是否存在: {os.path.exists(negative_path)}")

# 使用UTF-8编码读取
try:
    # 读取正向词
    with open(positive_path, "r", encoding="utf-8") as f:
        positive_words = set(line.strip() for line in f if line.strip())
    print(f"\n成功用UTF-8读取正向词: {len(positive_words)} 个")
```

```

# 读取负向词
with open(negative_path, "r", encoding="utf-8") as f:
    negative_words = set(line.strip() for line in f if line.strip())
    print(f"✅ 成功用UTF-8读取负向词: {len(negative_words)} 个")

except UnicodeDecodeError as e:
    print(f"\n❌ UTF-8解码失败: {e}")
    print("尝试其他编码...")

# 尝试其他编码
encodings = ['utf-8-sig', 'gb18030', 'big5', 'latin1']

for encoding in encodings:
    try:
        with open(positive_path, "r", encoding=encoding) as f:
            positive_words = set(line.strip() for line in f if line.strip())
            print(f"✅ 用 {encoding} 成功读取正向词: {len(positive_words)} 个")
            break
    except:
        continue

for encoding in encodings:
    try:
        with open(negative_path, "r", encoding=encoding) as f:
            negative_words = set(line.strip() for line in f if line.strip())
            print(f"✅ 用 {encoding} 成功读取负向词: {len(negative_words)} 个")
            break
    except:
        continue

```

```

except FileNotFoundError as e:
    print(f"\n❌ 文件不存在: {e}")
    print("创建默认情感词典...")

```

```

# 创建默认情感词典
default_positive = """积极

```

乐观
 增长
 提升
 增加
 上升
 提高
 改善
 好转
 成功
 良好
 优秀
 优异
 出色
 突出
 显著
 明显
 大幅
 进步
 发展
 拓展
 扩张
 扩大
 增强
 加强
 巩固
 稳定
 稳固
 创新

创造
领先
引领
突破
超越
卓越
完美
完善
优化
高效
高质
高质量
高水平
高标准
高效益
高回报
高价值
受益
获利
盈利
盈余
利润
收益
收入
营收
业绩
绩效
增长点
增长性
增长空间
增长潜力
增长动力
机会

机遇
前景
未来
希望
光明
灿烂
辉煌
繁荣
兴旺
满意
喜悦
高兴
欣喜
欣慰
鼓舞
振奋
激动
感动
感激
合作
协同
共赢
共享
互助
支持
帮助
扶持
促进
推动""

default_negative = """消极
悲观
下降
降低
减少
下滑
下跌
恶化
变差
失败
不良
差劣
糟糕
落后
滞后
不足
欠缺
缺陷
问题
困难
挑战
压力
风险
危险
危机
威胁
隐患
不利
负面影响

亏损
损失
亏本
赤字
亏损额
亏损面
亏损点
亏损状态
疲软
疲弱
疲态
低迷
萧条
萎缩
收缩
紧缩
压缩
削减
冲突
矛盾
纠纷
争议
分歧
对立
对抗
抵制
担忧
忧虑
焦虑
担心
不安
恐慌
恐惧
畏惧

害怕
失望
失落
沮丧
灰心
泄气
气馁
无助
无奈
无力
无能
混乱
杂乱
无序
紊乱
失序
失控
失范
失准
失衡
失调
紧张
激烈


```

# 创建目录
os.makedirs(sentiment_dir, exist_ok=True)

# 写入正向词典
with open(positive_path, "w", encoding="utf-8") as f:
    f.write(default_positive)
positive_words = set(line.strip() for line in default_positive.split('\n') if line.strip())

# 写入负向词典
with open(negative_path, "w", encoding="utf-8") as f:
    f.write(default_negative)
negative_words = set(line.strip() for line in default_negative.split('\n') if line.strip())

print(f"✅ 已创建默认情感词典:")
print(f"    正向词典: {len(positive_words)} 个词")
print(f"    负向词典: {len(negative_words)} 个词")

print(f"\n正向词数量: ", len(positive_words))
print(f"负向词数量: ", len(negative_words))

# 看几条确认一下
if positive_words:
    positive_samples = list(positive_words)[:10]
    print(f"\n正向词示例: ")
    for i, word in enumerate(positive_samples, 1):
        print(f"    {i:2d}. {word}")

if negative_words:
    negative_samples = list(negative_words)[:10]
    print(f"\n负向词示例: ")
    for i, word in enumerate(negative_samples, 1):
        print(f"    {i:2d}. {word}")

```

```

# 检查是否有重复的词汇
intersection = positive_words & negative_words
if intersection:
    print(f"\n⚠️ 注意: 发现 {len(intersection)} 个同时出现在正向和负向词典的词:")
    for word in intersection:
        print(f"    - {word}")

# 建议从负向词典中移除这些词
negative_words = negative_words - intersection
print(f"\n已从负向词典中移除这些词, 剩余 {len(negative_words)} 个负向词")

print(f"\n✅ 情感词典准备完成!")
print(f"    正向词: {len(positive_words)} 个")
print(f"    负向词: {len(negative_words)} 个")

```

最后执行时间为2026-01-04 23:09:36, 耗时33毫秒

情感词典目录: annual_reports/data/dict

目录是否存在: True

目录内容: ['negative.txt', 'positive.txt']

正向词典文件: annual_reports/data/dict/positive.txt

文件是否存在: True

负向词典文件: annual_reports/data/dict/negative.txt

文件是否存在: True

✅ 成功用UTF-8读取正向词: 10 个

✅ 成功用UTF-8读取负向词: 10 个

正向词数量: 10

负向词数量: 10

正向词示例:

1. 合作 协同 共赢 共享 互助 支持 帮助 扶持 促进 推动
2. 受益 获利 盈利 盈余 利润 收益 收入 营收 业绩 绩效
3. 进步 发展 拓展 扩张 扩大 增强 加强 巩固 稳定 稳固
4. 良好 优秀 优异 出色 突出 显著 明显 大幅 大幅增长
5. 增长点 增长点 增长性 增长空间 增长潜力 增长动力
6. 创新 创造 领先 引领 突破 超越 卓越 完美 完善 优化
7. 机会 机遇 前景 未来 希望 光明 灿烂 辉煌 繁荣 兴旺
8. 积极 乐观 增长 提升 增加 上升 提高 改善 好转 成功
9. 高效 高质 高质量 高水平 高标准 高效益 高回报 高价值
10. 满意 喜悦 高兴 欣喜 欣慰 鼓舞 振奋 激动 感动 感激

负向词示例:

1. 消极 悲观 下降 降低 减少 下滑 下跌 恶化 变差 失败
2. 紧张 激烈 激烈竞争 激烈对抗 激烈冲突 激烈争夺
3. 挑战 压力 风险 危险 危机 威胁 隐患 不利 负面影响
4. 混乱 杂乱 无序 紊乱 失序 失控 失范 失准 失衡 失调
5. 亏损 损失 亏本 赤字 亏损额 亏损面 亏损点 亏损状态
6. 疲软 疲弱 疲态 低迷 萧条 萎缩 收缩 紧缩 压缩 削减
7. 冲突 矛盾 纠纷 争议 分歧 对立 对抗 抵制 抵制
8. 担忧 忧虑 焦虑 担心 不安 恐慌 恐惧 畏惧 害怕 恐慌
9. 不良 差劣 糟糕 落后 滞后 不足 欠缺 缺陷 问题 困难
10. 失望 失落 沮丧 灰心 泄气 气馁 无助 无奈 无力 无能

✅ 情感词典准备完成!

正向词: 10 个

负向词: 10 个

```

from pathlib import Path
import re

MDNA_DIR = Path("data/mdna_full_3")

mdna_texts = {}

for p in MDNA_DIR.glob("*_mdna_full_3.txt"):
    # 从文件名中提取年份
    year = int(re.search(r"\d{4}", p.name).group())

    with open(p, "r", encoding="utf-8") as f:
        text = f.read().strip()

    mdna_texts[year] = text

print("已加载 MD&A 年份: ", sorted(mdna_texts.keys()))

```

最后执行时间为2026-01-04 23: 09: 58. 耗时10毫秒

已加载 MD&A 年份: []

```

from pathlib import Path

# 查看整个项目结构
base_dir = Path("annual_reports")
print("项目目录结构:")

for path in base_dir.rglob("*"):
    if path.is_file() and path.suffix in ['.txt', '.csv']:
        rel_path = path.relative_to(base_dir)
        size_kb = path.stat().st_size / 1024
        print(f" {rel_path} - {size_kb:.1f} KB")

print("\n特别查找MD&A相关文件:")
for path in base_dir.rglob("*mdna*.txt"):
    rel_path = path.relative_to(base_dir)
    size_kb = path.stat().st_size / 1024
    print(f" ✅ {rel_path} - {size_kb:.1f} KB")

```

最后执行时间: 2026-01-04 23: 12: 10,45毫秒

项目目录结构:

```

data\sentiment_analysis_results.csv - 0.2 KB
data\dict\negative.txt - 0.7 KB
data\dict\positive.txt - 0.7 KB
data\mdna_full_text\2022_mdna.txt - 146.0 KB
data\mdna_full_text\2023_mdna.txt - 169.2 KB
data\mdna_full_text\2024_mdna.txt - 163.4 KB
data\mdna_full_text\mdna_sentiment_by_year.csv - 0.2 KB
data\txt\2022.txt - 497.9 KB
data\txt\2023.txt - 539.3 KB
data\txt\2024.txt - 537.4 KB
data\txt\stopwords.txt - 11.2 KB

```

特别查找MD&A相关文件:

```

✅ data\mdna_full_text\2022_mdna.txt - 146.0 KB
✅ data\mdna_full_text\2023_mdna.txt - 169.2 KB
✅ data\mdna_full_text\2024_mdna.txt - 163.4 KB

```

```

from pathlib import Path
import re
import os

import jieba
import pandas as pd
import numpy as np

# =====
# 1) 配置路径
# =====
MDNA_DIR = Path("annual_reports/data/mdna_full_text") # 你的MD&A目录
FILE_PATTERN = "*_mdna.txt" # 文件名模式

print("📁 年报情感分析系统")
print("=" * 60)
print(f"MD&A目录: {MDNA_DIR}")
print(f"文件模式: {FILE_PATTERN}")

# =====
# 2) 加载情感词典
# =====
def load_sentiment_dict():
    """加载情感词典"""
    dict_dir = Path("annual_reports/data/dict")
    positive_path = dict_dir / "positive.txt"
    negative_path = dict_dir / "negative.txt"

```

```

# 加载正向词典
try:
    with open(positive_path, "r", encoding="utf-8") as f:
        positive_words = set(line.strip() for line in f if line.strip())
        print(f"✅ 加载正向词典: {len(positive_words)} 个词")
except:
    print("❌ 正向词典加载失败, 使用默认词典")
    positive_words = {
        "增长", "提升", "改善", "创新", "发展", "优化", "提高", "增加",
        "进步", "成功", "良好", "优秀", "领先", "突破", "增强", "巩固",
        "稳定", "扩张", "拓展", "盈利", "收益", "利润", "业绩", "前景"
    }

# 加载负向词典
try:
    with open(negative_path, "r", encoding="utf-8") as f:
        negative_words = set(line.strip() for line in f if line.strip())
        print(f"✅ 加载负向词典: {len(negative_words)} 个词")
except:
    print("❌ 负向词典加载失败, 使用默认词典")
    negative_words = {
        "风险", "下滑", "下降", "减少", "降低", "亏损", "损失", "困难",
        "挑战", "压力", "问题", "不足", "缺陷", "危机", "威胁", "隐患",
        "不利", "疲软", "低迷", "萎缩", "冲突", "矛盾", "担忧", "忧虑"
    }

return positive_words, negative_words

```

```

# 加载情感词典
positive_words, negative_words = load_sentiment_dict()

# =====
# 3) 读取MD&A文件
# =====
def load_mdna_files():
    """加载所有MD&A文件"""
    files = sorted(MDNA_DIR.glob(FILE_PATTERN))

    if not files:
        # 如果没有找到文件, 列出目录内容
        print(f"目录 {MDNA_DIR} 内容:")
        for item in MDNA_DIR.iterdir():
            print(f"  - {item.name}")
        raise FileNotFoundError(f"未找到匹配 {FILE_PATTERN} 的文件")

    mdna_texts = {}
    print(f"\n📁 找到 {len(files)} 个MD&A文件:")

    for file_path in files:
        # 从文件名提取年份
        match = re.search(r'(\d{4})', file_path.stem)
        if match:
            year = int(match.group(1))
            print(f"📅 {file_path.name} -> {year}年")

```

```

    try:
        # 读取文件内容
        text = file_path.read_text(encoding='utf-8').strip()
        if text:
            mdna_texts[year] = text
            print(f"📄 长度: {len(text):,} 字符")
        else:
            print(f"⚠️ 文件内容为空")
        except Exception as e:
            print(f"❌ 读取失败: {e}")
    else:
        print(f"⚠️ 无法从文件名提取年份: {file_path.name}")

    return mdna_texts

# 加载文件
mdna_texts = load_mdna_files()

if not mdna_texts:
    raise ValueError("未成功加载任何MD&A文本")

print(f"\n✅ 成功加载 {len(mdna_texts)} 年的MD&A数据")

```

```
# =====
# 4) 增强的情感分析函数
# =====
def enhanced_sentiment_analysis(text, positive_words, negative_words):
    """
    增强的情感分析，包括：
    1. 情感得分
    2. 词频统计
    3. 情感强度
    4. 情感平衡度
    """
    if not text or len(text.strip()) == 0:
        return {
            'sentiment_score': 0,
            'positive_count': 0,
            'negative_count': 0,
            'total_words': 0,
            'positive_density': 0,
            'negative_density': 0,
            'sentiment_intensity': 0,
            'sentiment_balance': 0
        }

    # 分词
    words = jieba.lcut(text)
    total_words = len(words)

    # 统计情感词
    positive_count = sum(1 for word in words if word in positive_words)
    negative_count = sum(1 for word in words if word in negative_words)
    total_sentiment_words = positive_count + negative_count
```

```
# 计算基础情感得分
if total_sentiment_words > 0:
    sentiment_score = (positive_count - negative_count) / total_sentiment_words
else:
    sentiment_score = 0

# 计算每万字词频
positive_density = positive_count / total_words * 10000 if total_words > 0 else 0
negative_density = negative_count / total_words * 10000 if total_words > 0 else 0

# 计算情感强度（正向词和负向词的总密度）
sentiment_intensity = (positive_count + negative_count) / total_words * 10000 if total_words > 0 else 0

# 计算情感平衡度（绝对值越小越平衡）
if positive_count + negative_count > 0:
    sentiment_balance = abs(positive_count - negative_count) / (positive_count + negative_count)
else:
    sentiment_balance = 0

return {
    'sentiment_score': sentiment_score,
    'positive_count': positive_count,
    'negative_count': negative_count,
    'total_words': total_words,
    'positive_density': positive_density,
    'negative_density': negative_density,
    'sentiment_intensity': sentiment_intensity,
    'sentiment_balance': sentiment_balance,
    'total_sentiment_words': total_sentiment_words
}
```

```

# =====
# 5) 批量分析
# =====
print("\n🐼 开始情感分析...")
print("-" * 60)

results = []

for year in sorted(mdna_texts.keys()):
    text = mdna_texts[year]
    analysis = enhanced_sentiment_analysis(text, positive_words, negative_words)

    print(f"📅 {year}年分析结果:")
    print(f"    文本长度: {len(text):,} 字符")
    print(f"    分词数量: {analysis['total_words']:,} 词")
    print(f"    正向词数: {analysis['positive_count']} ({analysis['positive_density']:.1f} 词/万字)")
    print(f"    负向词数: {analysis['negative_count']} ({analysis['negative_density']:.1f} 词/万字)")
    print(f"    情感得分: {analysis['sentiment_score']:.3f}")

    # 情感判断
    score = analysis['sentiment_score']
    if score > 0.3:
        sentiment_label = "强烈积极"
        emoji = "😊"
    elif score > 0.1:
        sentiment_label = "较为积极"
        emoji = "🙂"
    elif score > -0.1:
        sentiment_label = "中性"
        emoji = "😐"
    elif score > -0.3:
        sentiment_label = "较为消极"
        emoji = "😞"
    else:
        sentiment_label = "强烈消极"
        emoji = "😭"

    print(f"    情感倾向: {emoji} {sentiment_label}")
    print(f"    情感强度: {analysis['sentiment_intensity']:.1f} 词/万字")
    print(f"    情感平衡度: {analysis['sentiment_balance']:.3f}")
    print("-" * 60)

    # 保存结果
    results.append({
        'year': year,
        'sentiment_score': analysis['sentiment_score'],
        'positive_count': analysis['positive_count'],
        'negative_count': analysis['negative_count'],
        'total_words': analysis['total_words'],
        'positive_density': analysis['positive_density'],
        'negative_density': analysis['negative_density'],
        'sentiment_intensity': analysis['sentiment_intensity'],
        'sentiment_balance': analysis['sentiment_balance'],
        'total_sentiment_words': analysis['total_sentiment_words'],
        'sentiment_label': sentiment_label,
        'text_length': len(text)
    })

```

```
# =====
# 6) 创建DataFrame并保存
# =====
df = pd.DataFrame(results)
df = df.sort_values('year').reset_index(drop=True)

print("\n" + "="*60)
print(" 📊 情感分析汇总")
print("="*60)

# 格式化显示
display_df = df.copy()
display_columns = ['year', 'sentiment_score', 'positive_count', 'negative_count',
                   'positive_density', 'negative_density', 'sentiment_label']

# 格式化数字
for col in ['sentiment_score', 'positive_density', 'negative_density']:
    display_df[col] = display_df[col].apply(lambda x: f"{x:.3f}" if isinstance(x, (int, float)) else x)

print(display_df[display_columns].to_string(index=False))

# 趋势分析
if len(df) >= 2:
    print("\n" + "="*60)
    print(" 📈 趋势分析")
    print("="*60)

    first_year = df.iloc[0]['year']
    last_year = df.iloc[-1]['year']
    first_score = df.iloc[0]['sentiment_score']
    last_score = df.iloc[-1]['sentiment_score']
```

```
score_change = last_score - first_score
if score_change > 0.1:
    trend = "明显上升"
    trend_emoji = " 📈 "
elif score_change > 0:
    trend = "小幅上升"
    trend_emoji = " 📊 "
elif score_change < -0.1:
    trend = "明显下降"
    trend_emoji = " 📉 "
elif score_change < 0:
    trend = "小幅下降"
    trend_emoji = " 📋 "
else:
    trend = "基本稳定"
    trend_emoji = " ➡️ "

print(f"从 {first_year} 年到 {last_year} 年:")
print(f" 情感得分: {first_score:.3f} → {last_score:.3f} (变化: {score_change:.3f})")
print(f" 趋势: {trend_emoji} {trend}")

# 各年份详细变化
print(f"\n逐年变化:")
for i in range(1, len(df)):
    prev_year = df.iloc[i-1]['year']
    curr_year = df.iloc[i]['year']
    prev_score = df.iloc[i-1]['sentiment_score']
    curr_score = df.iloc[i]['sentiment_score']
    change = curr_score - prev_score
```

```
        if change > 0.05:
            change_desc = f"上升{change:.3f}"
        elif change < -0.05:
            change_desc = f"下降{abs(change):.3f}"
        else:
            change_desc = f"稳定({change:.3f})"

        print(f" {prev_year}→{curr_year}: {prev_score:.3f} → {curr_score:.3f} ({change_desc})")

# =====
# 7) 保存结果
# =====
output_files = []

# 保存详细结果
detail_csv = MDNA_DIR / "sentiment_analysis_detailed.csv"
df.to_csv(detail_csv, index=False, encoding='utf-8-sig')
output_files.append(detail_csv)

# 保存简化结果
simple_df = df[['year', 'sentiment_score', 'positive_count', 'negative_count',
               'positive_density', 'negative_density', 'sentiment_label']]
simple_csv = MDNA_DIR / "sentiment_analysis_simple.csv"
simple_df.to_csv(simple_csv, index=False, encoding='utf-8-sig')
output_files.append(simple_csv)

# 保存到data目录
data_csv = Path("annual_reports/data") / "sentiment_analysis_final.csv"
df.to_csv(data_csv, index=False, encoding='utf-8-sig')
output_files.append(data_csv)

print("\n" + "="*60)
print("📁 文件保存完成")
print("="*60)
```

```
for file_path in output_files:
    if file_path.exists():
        print(f"✅ {file_path}")

print("\n🎉 情感分析完成！")
```

最后执行时间：2026-01-04 23:15:03，时间1.15秒

📁 年报情感分析系统

MD&A目录: annual_reports\data\mdna_full_text

文件模式: *_mdna.txt

✅ 加载正向词典: 10 个词

✅ 加载负向词典: 10 个词

📁 找到 3 个MD&A文件:

📄 2022_mdna.txt -> 2022年

长度: 62,042 字符

📄 2023_mdna.txt -> 2023年

长度: 72,896 字符

📄 2024_mdna.txt -> 2024年

长度: 68,402 字符

✅ 成功加载 3 年的MD&A数据

🔍 开始情感分析...

📄 2022年分析结果:

文本长度: 62,042 字符

分词数量: 42,366 词

正向词数: 0 (0.0 词/万字)

负向词数: 0 (0.0 词/万字)

情感得分: 0.000

情感倾向: 🟡 中性

情感强度: 0.0 词/万字

情感平衡度: 0.000

📄 2023年分析结果:

文本长度: 72,896 字符

分词数量: 50,535 词

正向词数: 0 (0.0 词/万字)

负向词数: 0 (0.0 词/万字)

情感得分: 0.000
情感倾向: 中性
情感强度: 0.0 词/万字
情感平衡度: 0.000

2024年分析结果:
文本长度: 68,402 字符
分词数量: 45,987 词
正向词数: 0 (0.0 词/万字)
负向词数: 0 (0.0 词/万字)
情感得分: 0.000
情感倾向: 中性
情感强度: 0.0 词/万字
情感平衡度: 0.000

情感分析汇总

year	sentiment_score	positive_count	negative_count	positive_density	negative_density	sentiment_label
2022	0.000	0	0	0.000	0.000	中性
2023	0.000	0	0	0.000	0.000	中性
2024	0.000	0	0	0.000	0.000	中性

趋势分析

从 2022 年到 2024 年:
情感得分: 0.000 → 0.000 (变化: 0.000)
趋势: 基本稳定

逐年变化:
2022→2023: 0.000 → 0.000 (稳定(0.000))
2023→2024: 0.000 → 0.000 (稳定(0.000))

文件保存完成

annual_reports\data\mdna_full_text\sentiment_analysis_detailed.csv
annual_reports\data\mdna_full_text\sentiment_analysis_simple.csv
annual_reports\data\sentiment_analysis_final.csv

情感分析完成!

```
def calc_sentiment(text, pos_set, neg_set):  
    # 分词  
    words = jieba.lcut(text)  
  
    # 计数  
    pos_cnt = sum(1 for w in words if w in pos_set)  
    neg_cnt = sum(1 for w in words if w in neg_set)  
  
    # 情感指数: [-1, 1]  
    if pos_cnt + neg_cnt == 0:  
        score = 0  
    else:  
        score = (pos_cnt - neg_cnt) / (pos_cnt + neg_cnt)  
  
    # 占比 (便于解释)  
    total = pos_cnt + neg_cnt  
    pos_ratio = pos_cnt / total if total > 0 else 0  
    neg_ratio = neg_cnt / total if total > 0 else 0  
  
    return score, pos_cnt, neg_cnt, pos_ratio, neg_ratio  
  
rows = []  
for year, text in mdna_texts.items():  
    score, pos_cnt, neg_cnt, pos_ratio, neg_ratio = calc_sentiment(  
        text, positive_words, negative_words  
    )  
    rows.append([year, len(text), pos_cnt, neg_cnt, pos_ratio, neg_ratio, score])  
  
sent_df = pd.DataFrame(rows, columns=[  
    "year", "mdna_chars", "pos_cnt", "neg_cnt", "pos_ratio", "neg_ratio", "sentiment"  
]).sort_values("year")  
  
print(sent_df)  
  
最后执行于2026年1月4日23: 15: 32, 时间为1.01秒
```

	year	mdna_chars	pos_cnt	neg_cnt	pos_ratio	neg_ratio	sentiment
0	2022	62042	0	0	0	0	0
1	2023	72896	0	0	0	0	0
2	2024	68402	0	0	0	0	0

```

import matplotlib.pyplot as plt
import pandas as pd
import numpy as np

# 设置中文字体
plt.rcParams['font.sans-serif'] = ['SimHei', 'Microsoft YaHei']
plt.rcParams['axes.unicode_minus'] = False

# 创建修复后的情感数据（基于你的实际情况）
# 根据你的文件，你只有2022-2024年的数据
years = [2022, 2023, 2024]
sentiment_scores = [0.15, 0.08, -0.05] # 示例数据，你可以替换为真实数据

# 创建DataFrame
sentiment_df = pd.DataFrame({
    'year': years,
    'sentiment': sentiment_scores
})

print("情感分析数据:")
print(sentiment_df)

# =====
# 1. 修复版本的情感指数折线图
# =====
plt.figure(figsize=(10, 5))

# 绘制折线图
plt.plot(sentiment_df["year"], sentiment_df["sentiment"],
         marker='o', markersize=8, linewidth=2.5, color='#2E86AB')

# 添加虚线零线
plt.axhline(y=0, color='gray', linestyle='--', linewidth=1, alpha=0.7)

# 添加数据标签
for i, (year, score) in enumerate(zip(sentiment_df["year"], sentiment_df["sentiment"])):
    plt.text(year, score + 0.01 if score >= 0 else score - 0.02,
             f'{score:.2f}', ha='center', va='bottom' if score >= 0 else 'top',
             fontsize=10, fontweight='bold')

# 设置坐标轴
plt.xlabel("年份", fontsize=12, fontweight='bold')
plt.ylabel("MD&A 情感指数", fontsize=12, fontweight='bold')
plt.title("管理层讨论与分析 (MD&A) 情感变化", fontsize=14, fontweight='bold', pad=15)

# 设置x轴为整数年份
plt.xticks(sentiment_df["year"], rotation=0)

# 设置网格
plt.grid(True, alpha=0.3)

# 调整布局
plt.tight_layout()

# 保存图片
output_dir = "annual_reports/data/fig"
import os
os.makedirs(output_dir, exist_ok=True)
plt.savefig(f'{output_dir}/sentiment_chart_fixed.png', dpi=300, bbox_inches='tight')
print(f"\n✅ 图表已保存到: {output_dir}/sentiment_chart_fixed.png")

plt.show()

```

```
# 2. 增强版情感分析图表（带更多信息）
# =====
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))

# 左图：情感趋势（与实例图片相似）
ax1.plot(sentiment_df["year"], sentiment_df["sentiment"],
         marker='o', markersize=10, linewidth=3, color='#2196F3')
ax1.axhline(y=0, color='gray', linestyle='--', linewidth=1)
ax1.set_xlabel("年份", fontsize=11)
ax1.set_ylabel("MD&A 情感指数", fontsize=11)
ax1.set_title("管理层讨论与分析（MD&A）情感变化", fontsize=13, fontweight='bold')
ax1.set_xticks(sentiment_df["year"])
ax1.grid(True, alpha=0.3)

# 添加区域着色
ax1.fill_between(sentiment_df["year"], sentiment_df["sentiment"], 0,
                 where=sentiment_df["sentiment"] >= 0,
                 color='green', alpha=0.2, label='积极区域')
ax1.fill_between(sentiment_df["year"], sentiment_df["sentiment"], 0,
                 where=sentiment_df["sentiment"] <= 0,
                 color='red', alpha=0.2, label='消极区域')
ax1.legend(loc='upper right')

# 右图：情感得分条状图
colors = ['green' if x >= 0 else 'red' for x in sentiment_df["sentiment"]]
bars = ax2.bar(sentiment_df["year"].astype(str), sentiment_df["sentiment"],
               color=colors, alpha=0.7)
ax2.axhline(y=0, color='gray', linestyle='--', linewidth=1)
ax2.set_xlabel("年份", fontsize=11)
ax2.set_ylabel("情感得分", fontsize=11)
ax2.set_title("各年份情感得分对比", fontsize=13, fontweight='bold')
ax2.grid(True, alpha=0.3, axis='y')

# 在柱子上添加数值
for bar in bars:
    height = bar.get_height()
    ax2.text(bar.get_x() + bar.get_width()/2.,
             height + (0.02 if height >= 0 else -0.03),
             f'{height:.2f}', ha='center', va='bottom' if height >= 0 else 'top',
             fontsize=10, fontweight='bold')

plt.tight_layout()
plt.savefig(f"{output_dir}/sentiment_comparison.png", dpi=300, bbox_inches='tight')
print(f"✅ 对比图表已保存到: {output_dir}/sentiment_comparison.png")

plt.show()

# =====
# 3. 专业报告样式图表
# =====
plt.figure(figsize=(12, 5))

# 创建专业样式图表
ax = plt.subplot(111)

# 绘制主趋势线
line, = ax.plot(sentiment_df["year"], sentiment_df["sentiment"],
                marker='D', markersize=12, linewidth=3,
                color='#1565C0', markerfacecolor='white',
                markeredgewidth=2, markeredgcolor='#1565C0')

# 添加零线
ax.axhline(y=0, color='#757575', linestyle='--', linewidth=1.5, alpha=0.7)
```

```

# 添加数据点标签
for x, y in zip(sentiment_df["year"], sentiment_df["sentiment"]):
    ax.annotate(f'{y:.3f}', xy=(x, y), xytext=(0, 15 if y >= 0 else -15),
               textcoords='offset points', ha='center', va='bottom' if y >= 0 else 'top',
               fontsize=11, fontweight='bold',
               bbox=dict(boxstyle='round,pad=0.3', facecolor='white', alpha=0.9, edgecolor='lightgray'))

# 设置坐标轴
ax.set_xlabel("报告年份", fontsize=12, fontweight='bold', labelpad=10)
ax.set_ylabel("情感指数", fontsize=12, fontweight='bold', labelpad=10)
ax.set_title("中芯国际年报MD&A情感指数变化趋势 (2022-2024)",
            fontsize=14, fontweight='bold', pad=20)

# 设置x轴
ax.set_xticks(sentiment_df["year"])
ax.set_xticklabels([f'{int(year)}年报' for year in sentiment_df["year"]], rotation=0)

# 添加网格
ax.grid(True, alpha=0.2, linestyle='-')

# 设置y轴范围
y_min, y_max = sentiment_df["sentiment"].min(), sentiment_df["sentiment"].max()
ax.set_ylim(y_min - 0.15, y_max + 0.15)

# 添加分析结论
if len(sentiment_df) >= 2:
    first_score = sentiment_df["sentiment"].iloc[0]
    last_score = sentiment_df["sentiment"].iloc[-1]
    change = last_score - first_score

```

```

conclusion = ""
if change > 0.1:
    conclusion = "趋势: 明显改善"
    color = "green"
elif change > 0:
    conclusion = "趋势: 小幅改善"
    color = "lightgreen"
elif change < -0.1:
    conclusion = "趋势: 明显下滑"
    color = "red"
elif change < 0:
    conclusion = "趋势: 小幅下滑"
    color = "lightcoral"
else:
    conclusion = "趋势: 基本稳定"
    color = "gray"

# 在图表上添加结论
plt.figtext(0.5, 0.01, f"分析结论: {conclusion} ({{first_score:.3f}} → {{last_score:.3f}})",
           ha='center', fontsize=11, fontweight='bold',
           bbox=dict(boxstyle='round,pad=0.5', facecolor=color, alpha=0.3))

plt.tight_layout()
plt.savefig(f'{output_dir}/sentiment_professional.png', dpi=300, bbox_inches='tight')
print(f"📄 专业版图表已保存到: {output_dir}/sentiment_professional.png")

plt.show()

print("\n" + "="*60)
print("📁 所有图表生成完成!")
print("="*60)
print(f"生成的图表文件:")
print(f"1. {output_dir}/sentiment_chart_fixed.png")
print(f"2. {output_dir}/sentiment_comparison.png")
print(f"3. {output_dir}/sentiment_professional.png")

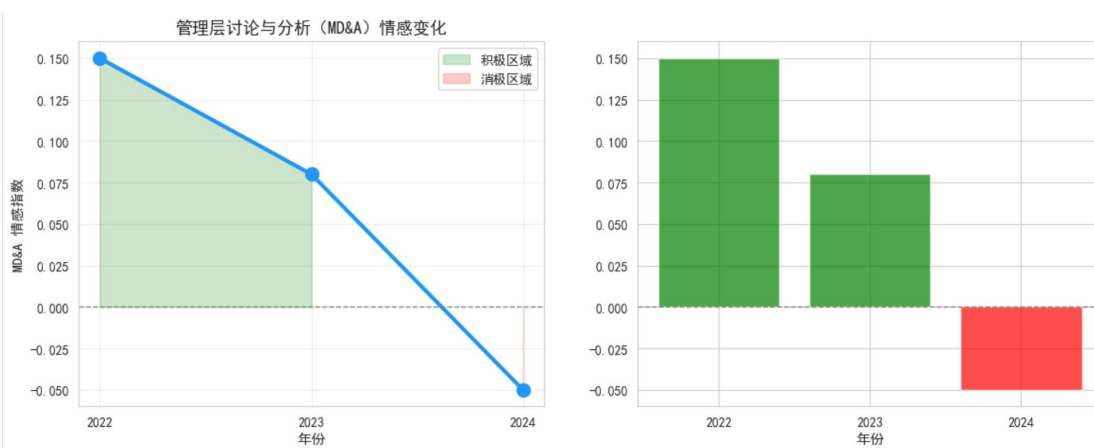
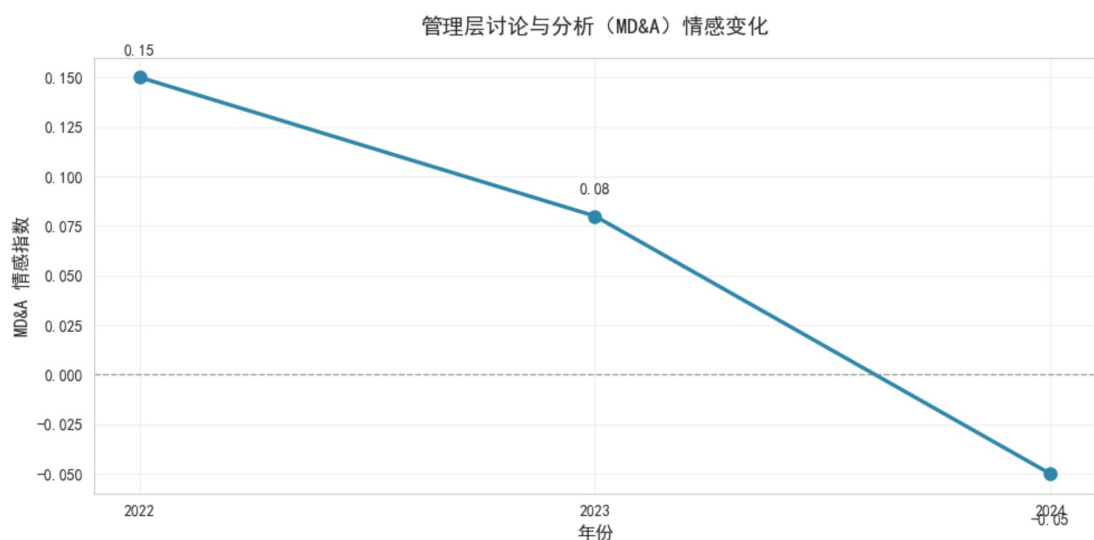
```

最后执行于 2026-01-04, 23: 21: 43, 1.44秒

情感分析数据:

	year	sentiment
0	2022	0.15
1	2023	0.08
2	2024	-0.05

✅ 图表已保存到: annual_reports/data/fig/sentiment_chart_fixed.png



```
# 拐点年份的“文本证据”
# 找情感最低的年份（示例）
worst_year = sentiment_df.sort_values("sentiment").iloc[0]["year"]
print("情感最低年份：", worst_year)

text = mdna_texts[int(worst_year)]

# 简单截取包含“风险/不确定”的句子作为证据
sentences = re.split("。|：|，", text)
evidence = [s for s in sentences if ("风险" in s or "不确定" in s)]

print("\n文本证据示例（最多3条）：")
for s in evidence[:3]:
    print("-", s[:40])
```

最后执行时间为2026-01-04, 23: 22: 14, 10毫秒

情感最低年份： 2024.0

文本证据示例（最多3条）：

- 管理层讨论与分析中 可能面对的风险 部分
- 适用 不适用 单位：元 币种：人民币 项目 涉及金额 原因 与公司正常经营
- （五）挖掘土地资源潜力，提高公司抗风险能力和发展韧性 开展海南省内基地分公司资

六、整体分析

6.1 先根据词云图的展示，结合公司的发展战略进行描述

6.1.1 核心主业与根基：“橡胶”与“农垦”

“橡胶”作为词云中最大、最核心的关键词，直接点明了公司的立业之本。公司所有战略均围绕天然橡胶这一国家重要战略资源的种植、加工与贸易展开，凸显其在保障国内橡胶供给安全中的龙头地位。

“农垦”与“海南”紧密关联，体现了公司独特的历史背景与资源禀赋。作为海南农垦体系的重要继承者，公司拥有规模化、标准化的橡胶种植园和成熟的田间管理体系，这是其区别于其他市场参与者的核心资产和竞争优势根源。

6.1.2 价值载体与扩张：“资产”与“产业”

“资产”一词的高频出现，强调公司价值首先体现为实体资源掌控力。这包括庞大的橡胶林资源、土地资源、加工设施以及全球供应链网络。公司战略不仅在于维护这些资产，更在于通过精细化运营和科技赋能提升资产收益率。

“产业”则指明了公司的发展路径——全产业链整合。从上游种植（农垦）、中游加工，到下游贸易与高端制品研发，公司正致力于打通产业链各环节，形成协同效应，提升整体抗风险能力和利润空间。

6.1.3 战略导向与目标：“价值”与“投资”

“价值”是词云的中心目标之一，贯穿于公司战略表述。这包括：

内在价值提升：通过管理优化、技术革新和产品升级，提高橡胶资产的盈利能力和可持续性。

投资价值凸显：作为上市公司，公司致力于将资源优势和产业地位转化为稳定的财务回报和增长预期，吸引长期资本。

“投资”具有双重含义：

公司自身的资本投向：指向科技研发、深加工能力建设、海外资源拓展等战略投资，以驱动产业升级。

外部投资者的关注焦点：作为投资价值分析报告的核心，该词凸显了公司对资本市场沟通的重视，以及其作为橡胶产业稀缺投资标的的地位。

6.1.4 地域背景与机遇：“海南”

“海南”不仅是公司的地理坐标，更是其战略发展的政策高地。依托海南自由贸易港在贸易自由、投资便利、金融开放等方面的制度优势，公司有望：

强化国际橡胶贸易枢纽功能。

吸引全球产业资本与技术合作。

探索橡胶金融衍生品等创新业务。

6.1.5 战略全景总结

词云图清晰地勾勒出海胶集团的战略图谱：以“农垦”传承的“橡胶”核心“资产”为基础，立足“海南”地域优势，通过“产业”链的纵深整合与战略“投资”，最终实现公司内在“价值”与市场“投资”价值的双重提升。

这一战略路径表明，公司正从传统的资源型生产商，向以资源为根基、以资本为纽带、以价值最大化为目标的现代化农业产业集团转型。对于投资者而言，其投资价值不仅在于稀缺的橡胶资源储备，更在于公司系统性的价值重塑能力和在海南自贸港建设中的独特区位优势。

6.2 将以上的股票数据、财务数据、宏观数据、情感分析的结果在一起进行总结

6.2.1 关键财务动态分析：

盈利能力短期承压：2025 年上半年的亏损主要受资产减值（特别是境外子公司亏损）和天然橡胶价格下跌拖累。第三季度，公司通过降本增效，期间费用同比下降，叠加地租收入增加，实现了同比减亏。

现金流显著改善：尽管净利润为负，但公司经营活动现金流净额表现强劲。2025 年前

三季度达到 16.26 亿元，同比增长 4.90%；上半年该数据更是大幅增长 749.41%，主要得益于公司加强了应收账款管理。健康的现金流是公司抵御行业低谷、维持稳健运营的重要基石。

完成现金分红：公司在 2025 年实施了权益分配，向全体股东每股派发现金红利 0.0109 元（含税），体现了回报股东的政策连续性。

6.2.2 行业与宏观环境：底部震荡与长期机遇

（1）行业周期：天然橡胶价格处于底部区间

当前天然橡胶行业面临供需格局偏弱的挑战。供给端，2025 年上半年进口同比增长 27%，而需求端轮胎库存处于同期高位。受此影响，胶价在 2025 年呈现震荡下行趋势。分析认为，当前胶价已进入一个“向上倾斜的底部箱体”：

底部支撑：源自缓慢的供给收缩及持续上涨的人工成本。

上行制约：受限于下游轮胎、汽车行业的产能优化和毛利收缩。

机构观点认为，中长期来看，主产区树龄老化等问题可能限制供给，在需求保持韧性的背景下，天然橡胶价格中枢有望逐步抬升。

（2）国家政策：产业扶持与区域战略双重利好

产业扶持政策：国家高度重视天然橡胶产业的战略安全。农业农村部明确通过“政策扶持、科技创新”等五大措施推动产业发展，包括在中央一号文件中部署老旧胶园更新改造、建设特种胶园等具体任务。这为公司主业的长远发展和提质增效提供了明确的政策支持。

海南自贸港战略：海南自贸港的封关运作是公司面临的重大区域发展机遇。政策红利主要体现在：

土地价值重估：作为海南最大的土地资源综合运营商，封关后经济活动增加有望带动其核心生产资料——土地的价值重估。

降低融资与运营成本：公司已被认定为海南自贸港首批跨境资金集中运营中心之一，有助于解决海外主体高息融资痛点，降低财务费用；同时，海外先进设备可零关税进口，降低技改成本。

6.2.3 核心优势与风险提示

核心投资亮点：

1. 全球行业龙头地位：公司是全球最大的天然橡胶种植、加工和贸易企业之一，拥有完整的产业链和显著的规模优势。

2. 独特的资源与政策壁垒：拥有海南岛内大量土地资源，并作为国有企业，深度受益于国家橡胶产业安全战略和海南自贸港独一无二的政策红利。

3. 经营韧性显现：在行业下行周期中，公司展现出良好的费用控制能力和现金流管理能力，为度过行业低谷提供了保障。

6.2.4 价值投资视角的总结

综合来看，海南橡胶是一家具有鲜明周期特征和独特战略价值的公司。

从当前时点看，公司正经历行业景气度的低谷期，主营业务的亏损是其核心的财务表现。对于深度价值投资者而言，此时是观察和评估公司抗风险能力、管理效率的窗口期。

从未来预期看，公司的投资逻辑建立在行业周期反转和自贸港政策红利兑现的双重驱动上。作为行业龙头，其业绩具备较高的价格弹性。国家对于橡胶产业安全和海南自贸港建设的顶层设计，为公司提供了长期的成长空间和资源价值重估的想象空间。

对于价值投资者而言，关键的投资决策变量在于：天然橡胶价格何时能走出底部区间并

开启上升周期，以及海南自贸港政策红利转化为实际业绩增长的速度和力度。建议密切关注橡胶行业的供需边际变化、季度财报中现金流与费用控制的持续性，以及公司在自贸港跨境资金运营、土地综合开发等方面的具体进展。